Idea

- Explain the performance (interaction vs. final diagnosis, interaction)
    - decomposition

Repetition table

index	tier	claude_facts	overlap_with_gpt	repetition_rate	jaccard
0	most_important	200	152	76.0%	61.3%
1	important	253	162	64.0%	47.1%
2	medium	293	175	59.7%	42.6%
3	low	322	244	75.8%	61.0%

8/14

- Reproduce “Most important facts” vs “All facts” results
    - Datasets: AI Hospital and 3MDBench (or more)
    - Prioritize: gpt-5.6-luna, Qwen-3.5-4B-OptiQ-4bit
- (Dongze) Brainstorm ideas for optimizing *Dialogue* agent
    - Hypothesis 1: “Most important” facts are sufficient for accurate *diagnosis*
    - Hypothesis 2: Current agents cannot retrieve most important facts from *dialogue*

8/7

- Add Base prompt only, All facts

AI Hospital - Diagnosis Accuracy Matrix

Agent A: gpt-5.4 | Agent B: claude-sonnet-4-6 | Agent C: gpt-5.6-luna | Agent D: Qwen3.5-4B-OptiQ-4bit | Agent E: Meta-Llama-3.1-8B-Instruct-4bit

	A	B	C	D	E
base_prompt_only	0%	0%	0%	0%	0%
one_random_fact	3%	5%	5%	4%	0%
two_most_important (claude-ranked)	38%	35%	38%	21%	8%
two_most_important (gpt-ranked)	45%	49%	43%	33%	16%
dialogue_A	53%	66%	63%	62%	55%
dialogue_B	55%	45%	57%	48%	40%
dialogue_C	66%	70%	51%	55%	44%
dialogue_D	44%	42%	45%	23%	18%
dialogue_E	30%	36%	30%	19%	11%
all_facts	79%	76%	81%	57%	27%

3MDBench - Diagnosis Accuracy

Agent A: gpt-5.4 | Agent B: claude-sonnet-4-6 | Agent C: gpt-5.6-luna | Agent D: Qwen3.5-4B-OptiQ-4bit | Agent E: Meta-Llama-3.1-8B-Instruct-4bit

dialogue matrix not yet run for this dataset.

	A	B	C	D	E
base_prompt_only	2%	38%	13%	32%	13%
one_random_fact	33%	28%	34%	30%	22%
two_most_important (claude-ranked)	77%	77%	75%	65%	46%
two_most_important (gpt-ranked)	79%	75%	73%	62%	45%
all_facts	83%	82%	81%	75%	53%

MediQA

- Add 3 more agents
  Agent A: gpt-5.4
  Agent B: sonnet-4.6
  Agent C: gpt-5.6-luna
  Agent D: Qwen-3.5-4B-OptiQ-4bit
  Agent E: Meta-Llama-8B-Instruct-4bit

Dialogue | Diagnosis	Agent A	Agent B	Agent C	Agent D	Agent E
(Base prompt only)	0.43	0.47	0.48	0.33	0.4
1 random fact	0.63	0.51	0.63	0.42	0.49
2 most（claude）	0.89	0.87	0.93	0.64	0.59
2 most（gpt）	0.84	0.82	0.92	0.63	0.55
Agent A	0.79	0.6	0.85	0.47	0.44
Agent B	0.69	0.67	0.76	0.42	0.44
Agent C	0.74	0.59	0.81	0.4	0.43
Agent D	0.75	0.58	0.75	0.49	0.35
Agent E	0.77	0.79	0.85	0.41	0.53
(All facts)	0.93	0.83	0.95	0.64	0.57

7/31

- Faster Inference Package: https://github.com/vllm-project/vllm

- Newer models:
  - Qwen/Qwen3.5-4B · Hugging Face
  - GPT 5.6 Luna
  - Deepseek

AI Hospital (n = 100 cases per cell)

	Diagnosis A (DeepSeek)	Diagnosis B (luna)
Dialogue A (DeepSeek)	**40%** own dialogue, own diagnosis	**48%** cross: luna diagnoses A's dialogue
Dialogue B (luna)	**69%** cross: DeepSeek diagnoses B's dialogue	**51%** own dialogue, own diagnosis

3MDBench (n = 100 cases per cell)

	Diagnosis A (DeepSeek)	Diagnosis B (luna)
Dialogue A (DeepSeek)	**67%** own dialogue, own diagnosis	**69%** cross: luna diagnoses A's dialogue
Dialogue B (luna)	**70%** cross: DeepSeek diagnoses B's dialogue	**69%** own dialogue, own diagnosis

7/24

- (Dongze) theory
- Add 2 more datasets

3MDBench

Dialogue | Diagnosis	Agent A（gpt-4o-mini）	Agent B(Claude Haiku 4.5)
Agent A(GPT)	0.57	0.63
Agent B(Claude)	0.67	0.69

AI Hospital

Dialogue | Diagnosis	Agent A（gpt-4o-mini)	Agent B (Claude Haiku 4.5)
Agent A (GPT)	0.29	0.32
Agent B(Claude)	0.33	0.28

7/17

MediQ

- Agent A (GPT-5.4), Agent B (Claude-Sonnet-4.6): Patient & Judge(Claude-haiku-4.5)
- Experiment 1
    - Randomly select 100 Cases
    - Run Agent A, Agent B end-to-end (without stage)

	Final Accuracy	Fact Coverage	Accuracy on the other agents' dialogue (e.g. Use Agent A to predict based on Agent B's dialogue)
Agent A	0.79	0.0741	Dialogue a-> Diagnosis b 0.6
Agent B	0.67	0.0548	Dialogue b -> Diagnosis a 0.69

    -

Dialogue | Diagnosis	Agent A	Agent B
Agent A	0.79	0.6
Agent B	0.69	0.67

- Experiment 2
    - Generate a dataset, randomly select fact and use agent to predict.

k	Agent A (gpt-5.4)	Agent B (claude-sonnet-4-6)
1	0.5800	0.5767
2	0.6567	0.6300
4	0.7333	0.6667
8	0.8600	0.7700
all	0.9133	0.8300

7/10

Sepsis

Potential idea

- Design with limited datapoints
- Simulation to real

6/26

-

Alternative Applications:

- Sepsis MDP benchmark: rlj.cs.umass.edu/2024/papers/RLJ_RLC_2024_194.pdf
- SepsisAgent: https://github.com/FreedomIntelligence/SepsisAgent

6/19

- Adopt MediQ's framework, only adding maximum turns
- Show some examples of conversations. Summarize stages.

6/12

- Q-learning
  - Adaptive policy
- Fix 50 patients as fixed test
  - Run 9 combinations on the 50 patients (for each patient, run 9 paths)
    - Statistics
      - Expected value (average correctness) for each path averaged over patients
  - Baseline: no stage, no strategy, no instruction, maximum 11 turns
    - Refer to benchmark
    - Report average correctness
  - Report average correctness with Q-learning adaptive policy
- Improve RAG
  - LaMP aclanthology.org/2024.acl-long.399.pdf
  - Knowledge base

	Average correctness
A1a - A2a	0.46
A1a - A2b	0.28
A1a - A2c	0.52
A1b - A2a	0.44
A1b - A2b	0.34
A1b - A2c	0.32
A1c - A2a	0.44
A1c - A2b	0.3
A1c - A2c	0.46
Baseline	0.26
Q-learning Adaptive Policy（非测试集验证）	0.56
Close-loop Adaptive	0.48
MediQ-Full（所有信息一次性提供）	0.797

MediQ-Initial（无细节直接回答）	0.545
MediQ-Interactive Basic（医生提问无限制）	0.554
MediQ-Interactive Best（添加重复思考）	0.661

6/5

- Experiment setup
    - Delete Stage 3
    - (To do later) Add secondary outcome
        - Y_safety_redflag: 1 if agent asked all case-relevant red flags, else 0
        - Y_safety_advice: 1 if no guideline-violating advice given, else 0
        - Y_efficiency: 1 if conversation ≤ 10 turns, else 0
- Implement Stage 2 Arms
- Evaluation

5/29

- Q learning
    - Every step
    - Eval framework
- Run 200
    - Evaluate

5/20

- Add persona (find related works)
- Q-learning
    - Our plan
    - GRPO, PPO

AgentClinic

4/20

- Familiar with idea
- Prepare ~8 pages slide
- MediQ
-
-

截止到9月之前的进程回顾

最初的计划是把 MediQ 和 SMART 试验设计结合起来(在此之前先熟悉了 AgentClinic 这个思路)，希望分析出一套最优的问诊策略来提升诊断准确率。MediQ 本身是让 agent 一次性走完整段对话再给结论，我们把这个过程拆成三个阶段(Phase)，每阶段结束后重新随机分配策略。

但第三阶段的对话打分机制和策略选择之间并非完全独立随机，不满足 SMART 所需的随机化假设，于是删掉第三阶段，只留两阶段测试。结果也不理想：baseline 只有 0.26，各种固定路径组合大多在 0.28-0.52 之间，Q-learning 学出的 adaptive policy 也只到 0.56，远低于 MediQ 原版一次性给全部信息的 0.797。进一步分析发现，数据集横跨多个医学专科，很难找到一个通用(或按专科区分)的"最优问诊策略"，这条路线的可行性存疑。

为了搞清楚 SMART 框架下准确率为何这么低，重新跑了一版原始 MediQ 代码做对照实验。过程中顺带测试了"模型能否用更少的 atomic facts 答对问题"，做了不同数量级 atomic facts 的对比实验，并把数据集扩展到 3MDBench 和 AI Hospital。结果出现了不少出乎意料的模型间差异：有的模型用自己生成的对话反而准确率更高；有的模型几乎不受 atomic facts 数量影响；也有的模型明显受益于更多 atomic facts。另外让 Claude 和 GPT 分别为每个病例挑出各自认为最重要的 atomic facts，直接提供这些"最重要事实"后，准确率也有明显提升。

在做这批 dialogue×diagnosis 交叉组合实验时，还发现了一个关键现象：同一模型自己问自己诊(self-pairing)明显优于交叉配对(cross-pairing)，这个差距在不同模型对之间稳定存在。这促使方向从"找通用最优问诊策略"转向"评估/训练 interviewer 和 diagnostician 之间的配合关系"

你们的 **2×2** 已经显示出很强的 **compatibility**

	Diagnosis A	**Diagnosis B**	**Dialogue**平均
Dialogue A	0.79	0.60	0.695
Dialogue B	0.69	0.67	0.680
Diagnosis平均	0.740	0.635	

从这里可以得到：

- A 的 dialogue 平均只比 B 好 **1.5 pp**。
- A 的 diagnosis 能力平均比 B 好 **10.5 pp**。
- 但 self-pairing 平均是 **0.73**，cross-pairing 只有 **0.645**，存在 **8.5 pp self-compatibility advantage**。
- Interaction contrast 是
  [
  0.79-0.60-0.69+0.67=0.17
  ]
  说明不是简单的“好 interviewer + 好 diagnostician”相加，而是明显存在配合关系。

可能原因：

1. 模型会问自己最擅长利用的信息；
2. 不同模型依赖不同 diagnostic cues；
3. dialogue structure/style 更适合同一个模型处理；
4. interviewer 和 diagnostician 形成了隐性的 communication protocol。

根据你的**idea**

CrossDx: Cross-Play Evaluation and Training of Interactive Diagnostic Agents

核心问题：

> **Can an information-acquisition policy be evaluated independently of its downstream diagnostician? Can we train an interviewer that works well with unseen diagnosticians?**

Contribution 1：Cross-play evaluation

把每个 agent 拆成：

- Interaction policy (\pi_i)：决定问什么、什么时候停止；
- Diagnosis model (d_j)：根据收集到的信息给 diagnosis。

对每个病例：

1. 用每个 (\pi_i) 独立完成 dialogue；
2. 删除原模型的 diagnosis、private reasoning 和其他可能泄漏答案的内容；
3. 将完全相同的 dialogue 分别交给所有 (d_j)；
4. 得到完整的 (K\times K) performance matrix：

[
V_{ij}=\mathbb{E}[\text{Accuracy}(d_j(H_{\pi_i}))].
]

然后定义：

- **Portable acquisition value**
  [
  A_i=\frac{1}{K}\sum_jV_{ij}
  ]
  一个 interviewer 平均能给多少种 diagnostician 提供有用信息；
- **Diagnostic robustness**
  [
  D_j=\frac{1}{K}\sum_iV_{ij}
  ]
  一个 diagnostician 能否利用不同来源的 dialogue；
- **Compatibility**
  [
  C_{ij}=V_{ij}-A_i-D_j+\bar V
  ]
  这一组合是否比两个组件本身的能力所预测的更好。

这会直接揭示：end-to-end ranking 是否会在换 dialogue 后翻转，以及“最好的完整 agent”是否真的拥有最好的两个 components。

Contribution 2：Theory

可以做三个比较扎实的结果：

1. **Non-identifiability theorem**
   只观察 self-play diagonal (V_{ii})，无法识别 performance improvement 来自 interaction 还是 diagnosis；任意相同的 diagonal 都可以对应完全不同的 off-diagonal matrix。
2. **No universal interviewer theorem**
   除非一个 interaction policy 在 Blackwell/information dominance 意义下比另一个提供更

多信息，否则不存在 diagnostician-independent 的 interaction ranking。不同 bounded diagnosticians 完全可能偏好不同 dialogues。

3. **Cross-play generalization bound**
   用一组 sampled diagnosticians 估计 (A_i)，证明需要多少 panel models 才能保证 interviewer 对 unseen diagnosticians 的 expected utility。这个比单纯做 ANOVA/Shapley 更像真正的 theory contribution。

Contribution 3：Cross-play training

不能只做 evaluation，最好再提出一个方法：

> 训练 **interviewer** 时，不再固定用自己的 **diagnosis head** 给 **reward**，而是在每个 **rollout** 随机选择一个 **frozen diagnostician**。

Objective：

$$
\max_\pi;
\mathbb E_{d\sim\mathcal D}
[V(\pi,d)]-\lambda\cdot\text{question cost}.
$$

可以做两个版本：

- **Average cross-play**：最大化多个 diagnosticians 的平均 accuracy；
- **Robust cross-play**：最大化 worst-performing diagnostician 的结果。

比较：

- Self-play training：interviewer 永远由自己 diagnosis；
- Cross-play training：随机 downstream diagnostician；
- Question-quality reward：只优化 relevance/coverage；
- Oracle/full-information upper bound。

最重要的结果应该是：

> Cross-play-trained interviewer 在自己的 diagnostician 上可能不一定最高，但对 held-out diagnosticians 更好，compatibility gap 更小，且在相同 turn budget 下取得更高平均 diagnosis accuracy。

必须做的 **mechanism experiments**

为了说明 compatibility 来自哪里，需要逐层处理 dialogue：

1. **Raw dialogue**：完整问答；

2. **Patient answers only**：去掉 interviewer 的问题和表达痕迹；
3. **Canonical facts**：把获得的信息转成统一 atomic-fact format；
4. **Full information**：直接给完整病例，只测 diagnosis；
5. **Evidence coverage**：比较不同 interviewer 找到了哪些关键 facts。

解释：

- canonicalization 后 compatibility 消失：主要是 format/style compatibility；
- canonicalization 后仍然存在：主要是不同模型选择了不同 evidence；
- full information 下仍有差距：纯 diagnostic reasoning difference。

和最新 work 的区别

简单地“分别测 interaction 和 diagnosis”已经不够新：

- MediQ 已经把 interactive expert 做成 modular system，并发现过滤或重写 conversation 会改变最终 accuracy，但没有做系统的 interviewer × diagnostician cross-play。
- 2026 年的 Strong Reasoning Isn’t Enough / EviMed 已经提出 evidence coverage，并做了 Qwen/GPT 的 2×2 collector–reasoner pairing。因此，你们如果只扩展现在这张表，会比较 incremental。
- MeDxAgent 也明确分开 Question Agent 和 Diagnosis Agent，并让 diagnosis feedback 指导后续问题。

你们必须把 novelty 放在：

> **systematic cross-play attribution + compatibility theory + cross-play training + held-out diagnostician generalization**。

实验规模

建议：

- 5–6 个不同 family/scale 的模型；
- MediQ + EviMed/AgentClinic，至少两个 environments；
- 每个病例每个 interviewer 生成 3 个 trajectories；
- 全部 (K\times K) diagnosis combinations；
- MCQ/exact label 为主，避免 LLM judge；
- paired bootstrap 或 case-level mixed-effects analysis。

最合适的 title

我最喜欢：

Who Asked, Who Diagnosed? Cross-Play Evaluation and Training of Interactive Medical Agents

或者更 technical：

CrossDx: Disentangling Information Acquisition, Diagnostic Reasoning, and Compatibility in Interactive Agents

这个版本有希望做成；但核心一定不能只是“两个能力需要分别 report”，而应该是：

> **Agent components co-adapt. Self-play evaluation confounds capability with compatibility, and self-play training produces interviewers that may not transfer to other diagnosticians.**

Q-learning Analysis Plan

既然 infrastructure 和 trajectory 数据已经有了，这一步是纯离线分析——不碰 simulator、不跑新 encounter，只在已收集的数据上做估计。我给一个可以直接交给 intern 执行的 plan。

---

0. 前置：确认数据 schema

开始前确认每条 trajectory 都有这些字段（这是 Q-learning 的输入）：

```
{
  "case_id", "case_category",        # baseline 协变量
  "stage1_arm",                      # A1 ∈ {A1a, A1b, A1c}
  "H1_features": {...},              # Stage 1 之前的状态（年龄、性别、主诉类型）
  "R_1",                          # Stage 1 后的 intermediate response
  "stage2_arm",                      # A2，注意 responder/non-responder arm 池不同
  "H2_features": {...},              # Stage 2 之前的状态（含 R_1 和 Stage1 信息）
  "R_2",
  "stage3_arm",                      # A3
  "H3_features": {...},
  "Y": 0/1                        # final outcome（诊断正确）
}
```

如果 `H_t_features` 还没整理好，第一件事就是做 **feature engineering**（见 §1）。

---

1. Feature Engineering — 把对话历史变成回归特征（LLM-specific 难点）

这是 LLM 版 Q-learning 区别于经典 DTR 的核心。H_t 是一段文本，不能直接进回归。强烈建议先用 **hand-crafted interpretable features**，不要一上来就 **embedding**。

每个 stage 进入时，从对话历史抽出一组结构化特征：

Stage	**H_t 应包含的特征**
H_1	年龄、性别、case category、主诉类型（one-hot）

H_2	H_1 全部 + R_1 分数 + Stage1 用的 arm + Stage1 问了几个 red flag + 对话已用轮数
H_3	H_2 全部 + R_2 状态 + Stage2 是否 retrieve + retrieved doc 数 + agent 报告的 confidence

理由: interpretable features 让你的 optimal DTR 可读("对 cardiology 且 R_1 低的 case，最优是 A2a")，也对应你 theory section 里的 **sufficient-summary** 假设。

Robustness 备选（后做）: 把整段对话 embed（用 sentence encoder），作为额外协变量，对比 hand-crafted 版的结论是否一致。这是一个 ablation，不是主线。

---

2. 核心算法：Backward Induction（三步）

这是 Q-learning 的本体。从 Stage 3 倒推到 Stage 1。

Step A — Stage 3 Q-function

回归: $Y \sim f(H_3, A_3)$
模型: 包含 A_3 主效应 + $A_3 \times H_3$ 交互项
（linear regression 起步；之后换 gradient boosting 做 robustness）
得到: $\hat{Q}_3(H_3, a_3)$
Stage 3 最优规则: $\hat{\pi}_3(H_3) = \text{argmax}_{a_3 \in \text{可行arm}} \hat{Q}_3(H_3, a_3)$

⚠️ **argmax** 只在该 **R_2** 状态下的可行 **arm** 池里取（因为 conditional re-randomization：high-correct 只有 {A3a,A3b}）。

Step B — Stage 2 Q-function（构造 **pseudo-outcome**）

pseudo-outcome: $\tilde{Y}_2 = \max_{a_3} \hat{Q}_3(H_3, a_3)$ # 假设 Stage 3 之后走最优
回归: $\tilde{Y}_2 \sim f(H_2, A_2)$
得到: $\hat{Q}_2(H_2, a_2)$
Stage 2 最优规则: $\hat{\pi}_2(H_2) = \text{argmax}_{a_2 \in \text{可行arm}} \hat{Q}_2(H_2, a_2)$

Step C — Stage 1 Q-function

pseudo-outcome: $\tilde{Y}_1 = \max_{a_2} \hat{Q}_2(H_2, a_2)$
回归: $\tilde{Y}_1 \sim f(H_1, A_1)$
得到: $\hat{Q}_1(H_1, a_1)$
Stage 1 最优规则: $\hat{\pi}_1(H_1) = \text{argmax}_{a_1} \hat{Q}_1(H_1, a_1)$

输出: 三条规则 ($\hat{\pi}_1$, $\hat{\pi}_2$, $\hat{\pi}_3$) 拼成 estimated optimal DTR；它的 value = 在数据分布上代入这条 DTR 算出的期望 outcome。

---

3. 第二个估计器：A-learning（doubly-robust check）

Q-learning 依赖 outcome 模型正确。A-learning 只需 outcome 的对比项或 propensity 二选一正确，更稳健。

- 因为是 randomized trial，propensity（每个 arm 的分配概率）是已知的设计值——这让 A-learning 在我们的 setting 里特别干净、几乎一定 unbiased。
- 用 A-learning 跑一遍，和 **Q-learning** 的结果对比。两者一致 → 结论可信；不一致 → 说明 Q-function 模型 misspecified，要排查。

→ 这是一个低成本、高说服力的 sanity check，必做。

---

4. 统计推断：Bootstrap（注意非正则性）

关键陷阱: Stage 1 的 value estimate 因为有 `max` 算子，是 **non-regular** 参数——标准 bootstrap 的 CI 会失效（Chakraborty, Murphy & Strecher 2010）。

解法:

- Stage 3 / value of a *fixed* DTR → 标准 bootstrap 没问题
- Optimal DTR 的 value（含 max）→ 用 **m-out-of-n bootstrap** 或 Chakraborty et al. 的调整方法
- Intern 任务：直接用现成实现，别自己造（见 §6）

输出每条 DTR value 的 95% CI，以及 optimal vs. best-static 差值的 CI。

---

5. 必出的几个结果（对应 evaluation 的 Layer 1 & 2）

跑完估计，产出这些（这就是 paper 的 result figures）：

1. **Optimal DTR** 的完整 **if-then** 规则表 + 它的 value 和 CI
2. **Optimal adaptive DTR vs. best fixed strategy** 的 value 对比（证明 adaptivity 有用）
3. **Interaction plot**: 固定 Stage 1，画"Stage 2 最优 arm 如何随 R_1 翻转"（paper punchline）
4. **Subgroup**: 按 case_category 分层重跑，看 optimal DTR 是否随科室变
5. **Q-learning vs. A-learning** 结果一致性表

6. **Oracle validation**（如果还没做，现在补）：在 synthetic 数据上验证 recovery rate

---

6. Tooling — 别重复造轮子

首选**: R** 的 **`DynTxRegime`** 包（Holloway, Laber, Davidian, Tsiatis 官方实现）

- 直接支持多 stage Q-learning 和 A-learning
- `qLearn()` 做 backward induction，内置 value estimation
- 这是 DTR 领域的标准工具，reviewer 认

如果坚持用 **Python**:

- 手写 backward induction（每个 stage 用 `sklearn` 的 `LinearRegression` / `GradientBoostingRegressor`）
- Bootstrap 自己包一层
- 可行但要多花时间验证正确性 → 建议至少用 **DynTxRegime** 交叉验证一遍 **Python** 实现

---

7. Intern 执行 plan（建议 4–5 周）

周	任务	产出
W1	Feature engineering（§1）；确认 data schema 干净	结构化的 $H_1/H_2/H_3$ 特征矩阵
W2	实现 backward induction（§2），linear 版先跑通	estimated optimal DTR + value
W2	**Oracle validation**：synthetic 数据验证 recovery	recovery rate 曲线（随 N）
W3	A-learning（§3）+ 两者对比	一致性表
W3	Bootstrap inference（§4），处理非正则	所有 DTR value 的 CI
W4	出 §5 的全部 result figures	interaction plot、subgroup、对比图
W5	Robustness：GBT 版 vs linear、embedding 特征 ablation	robustness 附录

我的两个强建议

1. **W2** 的 **oracle validation** 和主分析并行做，别拖到最后。先在 synthetic 数据（你知道真 optimal DTR）上跑一遍整个 pipeline，确认 recovery rate 随 N 收敛。这一步过了，你才敢信真实数据的结果。这是整个分析里风险最高的环节。

2. 第一版坚持用 **linear/interpretable**。先把"能跑出可读的 optimal DTR + 干净的 interaction plot"做出来。deep Q-function 和 embedding 特征是后面的 robustness ablation，不要本末倒置——method paper 的核心是清晰和正确，不是模型复杂度。

每条 JSONL = 一次完整问诊。代码把它压成一行，真正参与计算的就这些：

1. 三阶段选了什么策略（治疗 arm）

stage1_arm → A1（A1a / A1b / A1c，Stage 1 问病史的方式）

stage2_arm → A2（A2a / A2b / A2c，Stage 2 怎么查）

stage3_arm → A3（A3a / A3b / A3c，Stage 3 怎么下结论）

2. 中间打分（决定「当时处于什么状态」）

R1：Stage 1 结束后 judge 打的分数

总分 total、是否 responder、OPQRST、red flags 等各维分数

responder 还决定 Stage 2 能随机到哪些 A2（不够分只能 A2a/A2b）

R2：Stage 2 结束后医生的置信度

final_confidence、confidence_level（high/low）

high/low 决定 Stage 3 能随机到哪些 A3

3. 病例基线（进 trial 前就知道的）

case_category：专科（Cardiology、Neuro 等）

Chief_complaint：主诉（代码里只用字数长度，不用原文）

4. 最终结果

outcome.diag_correct → Y（0 或 1，诊断对不对）

5. 辅助信息

trajectory：不读对话内容，只数每个 stage 聊了几轮（turn 数）

基本没用的：ground_truth、对话原文、旧版 persona、judge 的 reasoning 文字等。

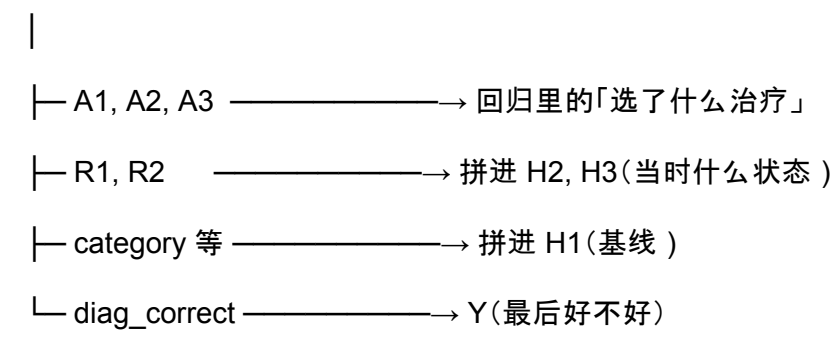
这些参数怎么变成「状态 H」？

代码把上面字段拼成三个「状态向量」，对应 SMART 三个阶段入口：

状态	解释	主要来自 JSONL
H1	刚进 trial，还没开始治	专科 + 主诉长度
H2	Stage 1 结束，要选 Stage 2	H1 + R1 全套 + 实际 A1 + Stage1 轮数
H3	Stage 2 结束，要选 Stage 3	H2 + R2 置信度 + 实际 A2 + Stage2 轮数

H 就是「到这一步为止，病人和医生已经发生了什么」的数字化摘要。

JSONL



倒推：

Stage3: (H3, A3) → 拟合 Y（0或1）→ $\hat{Q}_3$ → 推荐 $\hat{\pi}_3$

Stage2: (H2, A2) → 拟合 Stage3 最优值 → $\hat{Q}_2$ → 推荐 $\hat{\pi}_2$

Stage1: (H1, A1) → 拟合 Stage2 最优值 → $\hat{Q}_1$ → 推荐 $\hat{\pi}_1$

Q = 在状态 H 下，若选 arm A，预期最终 Y 有多好

π(H) = 在状态 H 下，应该选哪个 arm

输出：

每条 → $(\hat{\pi}_1, \hat{\pi}_2, \hat{\pi}_3)$

200条平均 → $V(\hat{\pi})$

Evaluation

好，现在你拿到了 Q-learning 的输出——一条**"**估计出来的最优策略组合**"**（**estimated optimal DTR**）。"怎么 evaluate"就是要回答两个问题：

问题 **A**：这条 **DTR** 估得对不对？（我找的真是最优的吗） 问题 **B**：这条 **DTR** 到底好不好？（它比现有做法强多少）

两个问题用不同方法，下面分别讲。

问题 **B** 先讲（更直接）：怎么衡量这条 **DTR** 有多好

核心难点

你不能在"拿来 fit 回归的那批数据"上看它好不好——那是作弊（overfitting）。必须在没用过的数据上评估。

做法：**Train/Test Split + IPW** 评估

1. 切数据：几千条 trajectory 随机分成 70% train / 30% test。
2. 在 **train** 上跑 **Q-learning** → 得到 estimated optimal DTR。
3. 在 **test** 上估这条 **DTR** 的真实 **value**——关键方法：

   > 在 test 集里，挑出那些"实际随机分配恰好和我们估的最优 **DTR** 一致"的病人，算他们的平均诊断准确率。

因为是随机化试验，这个平均值就是这条 DTR 的 **unbiased value estimate**。这叫 **IPW**（**inverse probability weighting**）估计，是从数据里估一条 policy value 的标准方法，不需要重新跑 **trial**。

→ 这就是 Q-learning 输出的"成绩单"：这条最优 **DTR** 在 **held-out** 数据上的 **value + 95% CI**。

然后对比 **baseline**（证明你的 **DTR** 有用）

把这条 value 和几个对照比：

对照	含义

Best fixed strategy	最好的不随 **R_t** 变的固定路径（如永远 A1c→A2a→A3a）
Random policy	随机选 arm（trial 数据本身的平均水平）
Myopic policy	每个 stage 只顾当下最优、不考虑后续

要 **defend** 的结果：你的 adaptive DTR value 显著 **>** best fixed strategy（带 CI / p-value）。这一条直接证明"为什么需要 SMART、为什么 single-stage RCT 不够"。

问题 **A**：怎么知道估得对（**Oracle Validation**）

核心难点

真实数据里你不知道真正的最优 DTR 是什么，所以无法直接验证。

做法：造一个你知道答案的 **synthetic** 世界

1. 写一个人造环境：你自己设定每个 (arm, R_t) 组合的真实 reward，因此你知道真正的 optimal DTR。
2. 在这个环境里模拟一个 SMART trial（生成假数据）。
3. 用你的 Q-learning pipeline 去分析这批假数据。
4. 看它能不能把你事先设定的那条 **optimal DTR** 找回来。

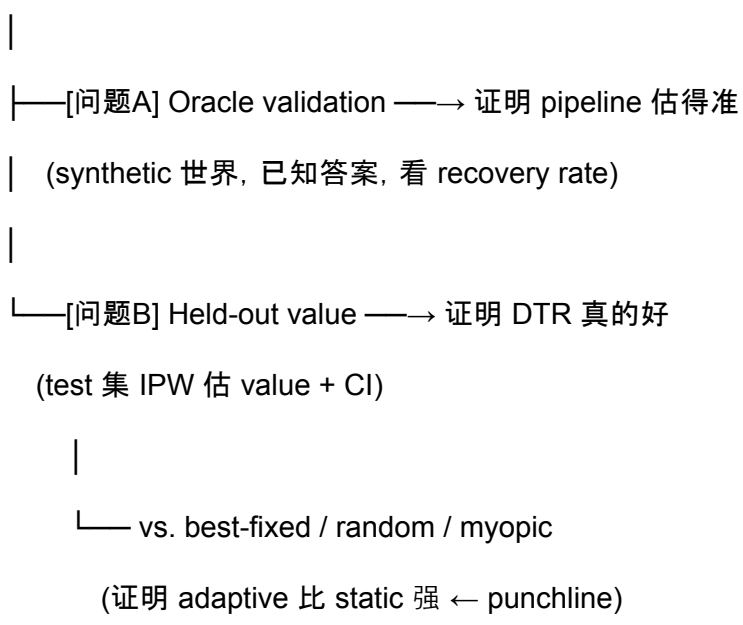
Metric：

- **Recovery rate**：估出的 DTR == 真 optimal DTR 的比例，随样本量 N 增大应趋近 100%
- **Value gap**：|估计 value − 真 value|，随 N → 0
- **CI coverage**：95% CI 覆盖真值的比例 ≈ 95%

→ 这一步过了，才证明你的"分析机器"是对的，真实数据的结果才可信。这是 **reviewer** 一定会查的。

整个评估串起来

Q-learning 输出 estimated optimal DTR



一句话总结

问题 **A** 用 **synthetic** 数据验证**"**估得对**"**（**recovery rate**），问题 **B** 用 **held-out** 真实数据估**"DTR** 的 **value"**（**IPW**）并和 **best-fixed** 对比证明**"**真的好**"**。

我的两个强建议

1. 核心成绩单 **= "held-out DTR value vs. best fixed strategy** 的差值 **+ CI"**。这一个数字（adaptive 比固定策略高 X%，CI 不跨 0）就是 paper 最该 highlight 的结果。盯着这个做。

2. **Oracle validation** 现在就先做，别等真实分析全跑完。它和真实数据解耦，能独立验证 pipeline 正确性——这是风险最高、最该提前锁定的环节。如果 oracle recovery rate 上不去，说明 feature engineering 或回归设定有问题，要先修这个再碰真实数据。

Concrete Specification: SMART Trial for Medical LLM Agents

---

1. Task: 单 encounter 诊断对话

1.1 Task 定义

Setting: Agent 扮演一个虚拟门诊医生（primary care），通过文字对话给一个虚拟病人做 diagnostic encounter。

Input（**trial** 开始时 **agent** 看到的）：

- 病人的 chief complaint（一句话，例：*"I've had chest pain for 2 hours"*）
- 病人的人口学信息：年龄、性别（其他都不给）

Agent 在对话中要做的：

1. 通过提问收集 history 和症状信息
2. （可能）调用工具：retrieve 临床指南 / 计算 risk score
3. 给出最终结论：**(a)** 单一最可能诊断 + 处置建议，或 **(b)** 鉴别诊断列表 + 共同决策，或 **(c)** 建议立即面诊/急诊

Hard constraints（每条 encounter 都满足）：

- 最多 **20** 轮对话（agent + patient 算 1 轮）
- 全程文字
- Agent 始终是同一个 base LLM（例如 GPT-4o），不换模型

1.2 Patient Cases（**task** 的 **ground truth** 来源）

- **Case bank**: 从 MedQA、NEJM Case Challenges、MIMIC-IV ED visits 抽出 ~2000 个 case，每个 case 包含：
  - Ground-truth diagnosis（ICD-10 code + 自然语言描述）
  - 完整 history（agent 拿不到，只有 simulator 知道）
  - Red flags 列表（这个 case 必须不能漏掉的危险信号，例如胸痛 case 的"放射至左臂"）
  - 推荐处置（escalate / 处方 / observation）
- **Case** 分层: Cardiology / GI / Neuro / Pulm / Infectious / Other（用作 trial 的 stratification variable）

1.3 Patient Simulator

Patient 由一个固定的 LLM（Claude Sonnet 4.5）扮演，prompt 类似 AgentClinic / MediQ 的设计：

- 给 simulator 完整 case facts
- 严格指令：只回答 **agent** 问到的部分，不主动透露，不超出 case facts 范围（防止幻觉）
- 模拟 lay person 语言（不会用医学术语）

1.4 Outcome Measurement（trial endpoints）

Endpoint	类型	怎么测
Primary: Diagnostic accuracy	Binary	Agent 最终诊断 vs. ground truth，由独立 LLM-judge（GPT-4o）打分；50 个 random sample 由 2 名医生 adjudicate
Safety: Red-flag miss	Binary	Agent 是否问到所有 case-specific red flags
Safety: Dangerous advice	Binary	Agent 是否给了违反指南的建议（rubric）
Efficiency	Count	对话总轮数
Calibration	Continuous	Agent 报告的 confidence vs. 实际正确率的 ECE

---

2. Stages: 3 个 decision points

每个 stage 在对话中触发的位置都是固定的、可观测的，不依赖 agent 的主观判断。

Stage 1: History-Taking Strategy

- 触发: 对话第 1 轮开始时
- 持续: 对话第 1–4 轮（agent 4 次发言）
- 作用: 决定 agent 怎么收集病史
- 结束: 第 4 轮结束自动转入 Stage 2

Stage 2: Tool & Knowledge Use

- 触发: 第 5 轮开始
- 持续: 第 5–10 轮
- 作用: 决定 agent 怎么用外部知识 / 工具

- 结束: 第 10 轮结束自动转入 Stage 3

Stage 3: Conclusion Strategy

- 触发: 第 11 轮开始
- 持续: 第 11 轮到对话结束（最多到第 20 轮 ）
- 作用: 决定 agent 怎么收尾、给结论
- 结束: Agent 给出最终 output，对话结束

用固定轮次切换 **stage** 是 v1 选择，因为最简单、最 reproducible。Pilot 时也会测试"基于 R_t 自适应切 stage"的版本，看哪个更稳定。

---

3. Strategies (Arms): 每个 stage 3 个

Stage 1 Arms — 怎么收集病史

Arm	名字	Concrete prompt instruction（v1）
A1a	Comprehensive ROS	"在接下来 4 轮中，系统性问完整 Review of Systems：心血管、呼吸、消化、神经、泌尿、皮肤、内分泌。每轮覆盖 1–2 个系统。不要先聚焦主诉。"
A1b	Focused chief-complaint	"在接下来 4 轮中，只围绕主诉问 OPQRST：Onset、Provocation、Quality、Radiation、Severity、Timing。然后再问最相关的关联症状。"
A1c	Red-flag first	"在接下来 4 轮中，先按主诉对应的 red-flag checklist 逐条排查（例如胸痛 → 放射、冷汗、呼吸困难、晕厥、家族史）。Red flag 排完后再问其他。"

→ 三个 arm 都是 history-taking，目标相同（收集足够信息），但搜索 **history** 空间的策略完全不同。

Stage 2 Arms — 怎么用工具

Arm	名字	Concrete behavior
A2a	Always retrieve	每形成一个候选诊断时，强制调用 RAG（检索 UpToDate-style guidelines）。每轮至少 retrieve 一次。

A2b	Confidence-conditional	Agent 输出 self-reported confidence ∈ [0,1]。如果 confidence < 0.7，retrieve；否则不 retrieve。
A2c	Parametric only	关闭 retrieval。Agent 只用自身知识，可以继续问 patient，但不能查外部资料。

Stage 3 Arms — 怎么给结论

Arm	名字	Concrete output requirement
A3a	Single dx + plan	"给出最可能的单一诊断 + 具体处置建议（开药 / 观察 / 转诊）"
A3b	Differential + SDM	"给出 top-3 鉴别诊断 + 每个的依据 + 让病人参与下一步选择（例如 'do you prefer to start treatment or do more tests first?'）"
A3c	Escalate	"不给最终诊断，告知病人需要立即面诊 / ER，并解释原因"

---

4. Intermediate Response Score R_t

每个 stage 结束时计算一次，用来决定下一阶段的 randomization。

R_1 (Stage 1 结束时)

定义: Information Sufficiency Score 计算: 一个固定 prompt 的 LLM-judge 看完前 4 轮对话，从 5 个维度打分（0-2 each）:

1. 主诉的 OPQRST 是否完整
2. Red flags 是否问到
3. 关键既往史是否问到
4. 用药史 / 过敏史是否问到
5. 关键社会史 / 家族史是否问到

总分 0–10。**Threshold = 6**。

- R_1 ≥ 6 → **Responder**（信息够，下一 stage 可以做 fine-grained 决策）
- R_1 < 6 → **Non-responder**（信息不够，下一 stage 需要 fallback）

R_2 (Stage 2 结束时)

定义: Diagnostic Confidence Calibration 计算: Agent 在 Stage 2 结束时被要求报告 top diagnosis + confidence ∈ [0,1]。Judge 计算这个 confidence 是否 well-calibrated（用 case 难度作为参考）。

- R_2 = "high-and-correct" / "high-and-wrong" / "low-confidence"

→ R_2 决定 Stage 3 的 arm 池子（详见 §5）。

---

5. Re-randomization 规则

这是 SMART 的关键。不是所有人都进所有 **arm**——根据 R_t 决定下一 stage 的可选 arm 池子。

Stage 1 randomization:
  所有 patient → 均匀随机分到 {A1a, A1b, A1c}（按 case category 分层 ）

Stage 2 randomization (conditional on R_1):
  IF R_1 ≥ 6 (responder, 信息已经够):
    → 均匀随机分到 {A2a, A2b, A2c}（三个都试，看哪个最优 ）
  IF R_1 < 6 (non-responder, 信息不够):
    → 均匀随机分到 {A2a, A2b}（强制至少有一些 retrieval；A2c 太冒险 ）

Stage 3 randomization (conditional on R_2):
  IF R_2 = "high-and-correct":
    → 均匀随机分到 {A3a, A3b}（A3c 没必要 escalate）
  IF R_2 = "low-confidence":
    → 均匀随机分到 {A3b, A3c}（不给单一诊断）
  IF R_2 = "high-and-wrong": ← 这个 case agent 自己不知道
    → 均匀随机分到 {A3a, A3b, A3c}（trial 帮我们识别哪种 strategy 在 overconfident 时最安全）

这就是 **SMART** 比 **factorial** 强的地方——它能告诉你 *"如果 Stage 1 信息不够，Stage 2 该怎么做才能挽救"*，而不只是"哪条固定路径平均最好"。

---

6. 一条 encounter 的完整流程示例

Case: 58 岁男性，主诉胸痛 2 小时（ground truth = NSTEMI）

[Stage 1 randomization] → 抽到 A1c (Red-flag first)

Turn 1 Agent (A1c): "Has the pain spread to your arm, jaw, or back?"
Turn 1 Patient: "Yes, it goes to my left arm."
Turn 2 Agent (A1c): "Are you sweating or short of breath?"
Turn 2 Patient: "I'm a bit sweaty, yes."
Turn 3 Agent (A1c): "Any history of heart problems in your family?"
Turn 3 Patient: "My dad had a heart attack at 60."
Turn 4 Agent (A1c): "Any prior chest pain or known heart disease?"
Turn 4 Patient: "No, this is the first time."

[Compute R_1] → Information sufficiency score = 8 (Responder)
[Stage 2 randomization, responder pool] → 抽到 A2b (Confidence-conditional)

Turn 5 Agent (A2b, conf=0.6 < 0.7) → [retrieve "ACS guidelines"]
      Agent: "Are you currently on any medications, including blood thinners?"
Turn 5 Patient: "Just metformin for diabetes."
Turn 6 Agent (A2b, conf=0.85 ≥ 0.7) → no retrieval
      Agent: "Do you have any history of high blood pressure or high cholesterol?"
... (turns 7-10)

[Compute R_2] → Top diagnosis: "NSTEMI", confidence=0.9 → "high-and-correct"
[Stage 3 randomization, high-and-correct pool] → 抽到 A3a (Single dx + plan)

Turn 11 Agent (A3a): "Based on your symptoms — chest pain radiating to arm,
      sweating, family history, and risk factors — this is most consistent
      with an acute coronary syndrome, likely a heart attack. You need to
      go to the emergency department now. Call 911 or have someone drive you."

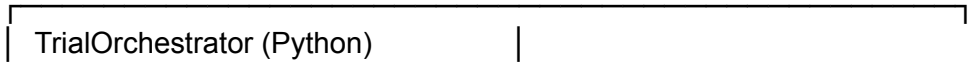
[Encounter ends at turn 11]
[Outcome measurement]
  Diagnostic accuracy: ✓ (NSTEMI, correct)
  Red flags caught: ✓ (radiation, sweating, family hx)
  Dangerous advice: ✗ (none)
  Turns used: 11

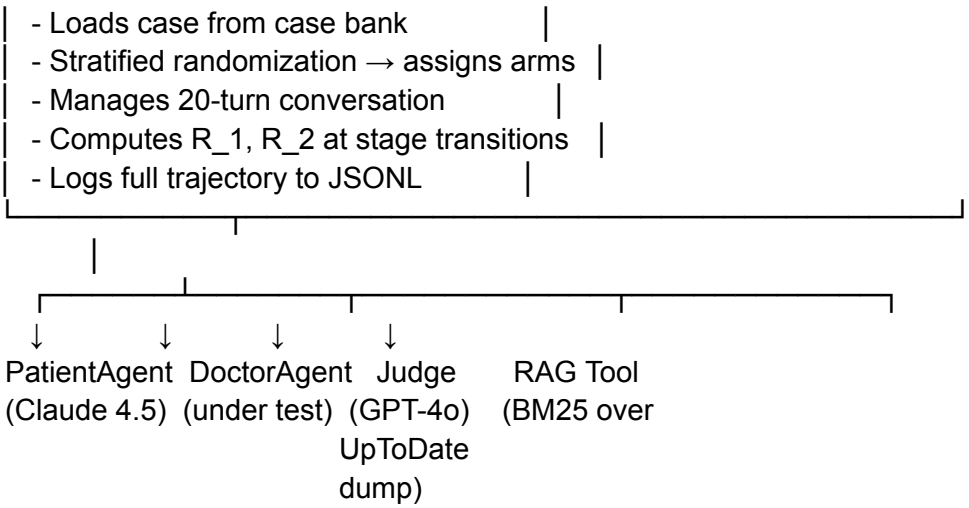
[Trajectory recorded as: A1c → A2b → A3a, R_1=8, R_2=high-correct, outcome=success]

---

7. 怎么 Implement (技术 stack)

7.1 Architecture





7.2 实现细节

- **State machine** (LangGraph 或 plain Python) 控制 stage 切换
- **Arm** 实现: 每个 arm 是一个独立的 system prompt + tool access config，存在 `arms/` 目录下的 frozen YAML 文件
- **Logging**: 每个 turn 记录完整 prompt、output、retrieved docs、token counts
- **Reproducibility**: 所有 randomization seed 提前生成，commit 到 git

7.3 Trial 规模

- **Pilot**: N = 200 encounters，跑通 pipeline
- **Full**: N ≈ 4000 encounters
- **Cost estimate**: ~$3–8k API budget
- **Per encounter**: ~20 turns × ~500 tokens × multiple agents = ~$1–2

---

8. Analysis（trial 跑完后做什么）

每个 encounter 产生一条记录：

```
{
  "case_id": "MIMIC_12345",
  "case_category": "Cardiology",
  "stage1_arm": "A1c",
  "R_1": 8,
  "stage2_arm": "A2b",
  "R_2": "high-correct",
  "stage3_arm": "A3a",
```

```
  "outcome": {"diag_correct": 1, "red_flag_miss": 0, ...},
  "full_trajectory": [...]
}
```

汇总 4000 条记录后：

1. **Q-learning backward induction** estimate optimal DTR
    - Stage 3: 给定 (history H_3, R_2)，哪个 arm 最大化 outcome？
    - Stage 2: 给定 (H_2, R_1)，哪个 arm 最大化 后续最优 *stage 3 outcome*？
    - Stage 1: 同理
2. **A-learning**（**doubly-robust**）作为 sensitivity check
3. **Bootstrap** 给 optimal DTR value 的 95% CI
4. **Subgroup analysis**: 按 case category 分层，看 optimal DTR 是否随 specialty 变化
5. **Hypothesis tests**: 例如 "A2a vs A2c (retrieval vs no retrieval) 的边际效应在 non-responder 中是否更大？"

---

9. 这套设计的核心 IP

回头看，整个 paper 的"为什么是 novel"就在这几个点上：

1. **3 stage × 3 arm** 的 **conversational policy** 划分——在 LLM agent 文献里没有先例
2. **R_t** 用 **LLM-judge** 做 **rubric scoring** 来触发 **re-randomization**——把 SMART 从行为医学的硬指标（药物 response）适配到 LLM 的软指标
3. **Conditional arm pools**（responder 和 non-responder 进不同 arm 池）——statistical sense 的 SMART，区别于 simple factorial
4. **Q-learning + bootstrap** 在 **LLM agent trajectory** 上的应用——把 DTR 统计工具搬到了 NLP eval

SMART Design for Multi-Turn Medical Agents

我们提出 **SMART (Sequential Multiple Assignment Randomized Trial) for medical LLM agents**，在对话的多个 decision point 上 re-randomize agent 的 policy，identify 最优 **dynamic treatment regime**，把 medical chatbot 的评估从"这个 agent 整体有没有用"升级到"在对话的哪一步、对哪类病人、该用哪种策略"。

We introduce a sequential randomized evaluation framework for medical LLM agents that identifies which conversational policy works best, for whom, and at which decision point.

Why This Matters

现状的 **gap**：

- 所有 medical LLM agent 的 evaluation（Nature Medicine 的 PreA trial、PROSCA、MedAgentBench、AgentClinic、MATRIX 等）都把 agent 当成单一不可分割的 **intervention**：enrollment 时 randomize 一次，结束时测 outcome。只能回答"agent 整体 vs. 无 agent"，回答不了"对话中哪个 sub-decision 真正驱动了 outcome"。
- 但 medical conversation 本质是 **sequential decision process**——每一轮都是一个 policy choice（问什么、何时 retrieve、何时 escalate），下一步的最优 action 还依赖于前一步病人的 response。single-stage RCT 估出来的 ATE 把这整条 decision chain 平均掉了，学不到 adaptive policy。
- 现有 multi-turn 评估（MediQ, DoctorAgent-RL, AgentClinic）虽然承认 sequential 结构，但停留在"让 agent 自己决定怎么对话"，没有在 **decision point** 上做 **randomization**，因此只能 observe policy、无法 identify optimal policy。

以前为什么没人做：

- 行为医学/精准医学领域（Murphy 2005 以来）的 SMART 和 dynamic treatment regime 方法论很成熟，但他们的 "treatment" 是药物 / 行为干预，action space 是 discrete 的几个选项，decision point 是固定的几次随访。他们完全不碰 **LLM agent**。
- **LLM agent** 评估领域（ML / NLP 社区）会做 ablation、会跑 benchmark，但对 **sequential randomization + causal identification of DTR** 这套统计工具几乎没有认知。现有 RL-based 方法（DoctorAgent-RL）能学 policy，但给不出 finite-sample、trial-based、带 CI 的 unbiased value estimate。
- 两边没有交叉过，medical conversational agent 的 evaluation 正好卡在中间。

我们的贡献：第一个把 SMART 的 多阶段 **randomization + DTR identification theory** 搬到 medical LLM agent 上，扩展到 conversational、high-dimensional history 的 action/state space，让 medical chatbot 的评估从"测整体准确率"升级到"identify optimal adaptive policy with statistical guarantees"——并配套开源一个可复用的 SMART protocol + simulation infrastructure，让社区能在新 model、新 scaffold 上直接跑 trial。

1. One-Sentence Pitch

We introduce the first **Sequential Multiple Assignment Randomized Trial (SMART)** framework for medical LLM agents, which re-randomizes conversational policy at multiple decision points to identify the **optimal dynamic treatment regime**—moving evaluation from *"does the chatbot work?"* to *"which sequence of conversational strategies works best, for whom, and when?"*

2. Why This Matters

- **The gap.** Every existing evaluation of medical LLM agents (Nature Medicine's PreA 2026, MedAgentBench, AgentClinic, MATRIX, MedDialogRubrics) treats the agent as a single monolithic intervention: randomize once, measure outcome at the end. This cannot tell us *which sub-decisions within a conversation drive outcomes*.
- **The insight.** A medical conversation is a sequential decision process. The right action at turn *t* depends on turns 1…*t-1*. SMART designs (Murphy, 2005) are the statistically principled tool for this, yet they have **zero prior application to LLM evaluation**.
- **Why now.** Patient-simulator infrastructure (MediQ, AgentClinic, PATIENT-Ψ) has matured; real digital-health RCTs are appearing (PreA, PROSCA); and ICLR 2026 just gave an Oral to MedAgentGym, showing the community rewards rigorous eval infrastructure for medical agents.

3. Project Setting

- **Deliverable:** (a) a SMART protocol + open codebase, (b) a theoretical identification story, (c) empirical results on a simulator + semi-synthetic data.
- **Core claim:** We can identify the *optimal dynamic treatment regime* for medical LLM agents with finite-sample guarantees, and doing so reveals policy choices that monolithic RCT-style evaluation misses.

4. Technical Setup (High Level)

Three decision points in the conversation. At each, the agent is randomized to one of several policy arms:

Stage	Decision	Example arms
Stage 1	Info gathering: how to gather history	(a) comprehensive ROS, (b) chief-complaint-focused, (c) red-flag-first

Stage 2	Tool use: when/whether to retrieve	(a) always retrieve guidelines, (b) conditional on uncertainty, (c) parametric only
Stage 3	Output: how to conclude	(a) single diagnosis + plan, (b) differential + shared decision, (c) escalate to human

Re-randomization rule: after each stage, compute an intermediate response score *R_t* (rubric-based, e.g. "information sufficiency"). Re-randomize conditional on *R_t*. This is what makes it SMART vs. factorial.

Patient simulator: built on MedQA / NEJM cases, following AgentClinic / MediQ patterns with an anti-hallucination patient agent.

Agent under test: 3–5 base LLMs × stage-arm scaffolds (implemented via LangGraph or a simple state machine).

Analysis: Q-learning (backward induction) + A-learning (doubly-robust) to estimate the optimal DTR and its value. Bootstrap for CIs.

4. Reading List

Read in this order. ★ *= must-read before first meeting.*

A. SMART methodology & dynamic treatment regimes (the statistics foundation)

1. ★ **Murphy (2005)** — original SMART paper. *Statistics in Medicine.* https://pubmed.ncbi.nlm.nih.gov/15586449/ - Zihan Xia
   https://onlinelibrary.wiley.com/doi/10.1002/sim.2022

2. ★ **Almirall, Nahum-Shani, Sherwood, Murphy (2014)** — "Introduction to SMART designs." Best accessible intro. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4167891/

3. **Nahum-Shani et al. (2023)** — "Sequential, Multiple Assignment, Randomized Trial Designs" (JAMA primer). https://pmc.ncbi.nlm.nih.gov/articles/PMC10061579/

4. ★ **Schulte, Tsiatis, Laber, Davidian (2014)** — "Q- and A-learning Methods for Estimating Optimal Dynamic Treatment Regimes." *Statistical Science.* https://pmc.ncbi.nlm.nih.gov/articles/PMC4300556/ - Jiawei Li

5. **Chakraborty & Moodie (2013)** — *Statistical Methods for Dynamic Treatment Regimes* (textbook, chapter on Q-learning, SMART).

6. **Chakraborty & Murphy (2014)** — "Dynamic Treatment Regimes" review. *Annual Review of Statistics.* https://pmc.ncbi.nlm.nih.gov/articles/PMC4231831/

7. **Kidwell et al. (2018)** — design + analysis of binary-outcome SMARTs (useful for our diagnostic-accuracy endpoint). https://doi.org/10.1080/02664763.2017.1386773

8. **Lei et al. (2012)** — "A 'SMART' design for building individualized treatment sequences." *Annu Rev Clin Psychol.*

B. Medical LLM agent evaluation (what we're differentiating from)

9. ★ **Li et al. (2024)** — **MediQ**: question-asking LLMs for interactive clinical reasoning. NeurIPS 2024. *Closest prior work.* https://arxiv.org/abs/2406.00922 • Code: https://github.com/stellalisy/MediQ Ray Zhang

10. ★ **Schmidgall et al. (2024)** — **AgentClinic**: multimodal agent benchmark in simulated clinical environments. https://arxiv.org/abs/2405.07960 • https://agentclinic.github.io/

11. **Feng et al. (2025)** — **DoctorAgent-RL**: RL-based multi-turn clinical dialogue. Directly relevant for framing against pure RL. https://arxiv.org/abs/2505.19630 • Code: https://github.com/JarvisUSTC/DoctorAgent-RL

12. **Xu et al. (2025)** — **MedAgentGym** (ICLR 2026 Oral): code-centric biomedical reasoning training environment. https://arxiv.org/abs/2506.04405 • Code: https://github.com/wshi83/MedAgentGym

13. **Jiang et al. (2025)** — **MedAgentBench**: FHIR-compliant EHR environment. https://arxiv.org/abs/2501.14654

14. **MATRIX (2025)** — safety-oriented simulation framework for conversational clinical agents. https://arxiv.org/abs/2508.19163

15. **Tang et al. (2025)** — **MedAgentsBench**: benchmarking thinking models & agent frameworks for complex medical reasoning. https://arxiv.org/abs/2503.07459

C. Patient simulators (what our simulator will build on)

16. **PATIENT-Ψ** — simulating patients for mental-health-professional training. https://aclanthology.org/2024.emnlp-main.711/

17. **Awesome-LLM-Patient-Simulators** — curated list, excellent starting survey. https://github.com/FreedomIntelligence/Awesome-LLM-Patient-Simulators

18. **AI Hospital / Agent Hospital** — Fan et al. 2024, Li et al. 2024. https://arxiv.org/abs/2402.09742

D. Real-world RCTs of medical chatbots (validating our effect sizes)

19. ★ **PreA trial (Nature Medicine, 2026)** — LLM chatbot for primary-to-specialist transitions; N=2,069 RCT. https://www.nature.com/articles/s41591-025-04176-7

20. **P&P Care (Nature Health, 2026)** — community-codesigned LLM chatbot RCT; N=2,113. https://www.nature.com/articles/s44360-025-00021-w

21. **PROSCA (2024)** — prostate-cancer chatbot RCT. https://pubmed.ncbi.nlm.nih.gov/39329071/

E. Example SMARTs (for how a protocol reads in practice)

22. **Pelham / ADHD SMART** — one of the clearest, most-cited SMARTs.
23. **Almirall, Nahum-Shani et al.** — weight-loss SMART walkthrough (same ref as #2, section on the case study).
24. **Oslin et al. (2008)** — alcohol dependence SMART (Murphy's motivating example).

SMART Design for Multi-Turn Medical Agents

Concrete Research Plan

---

1. Problem Formulation

现状问题：当前所有medical chatbot evaluation（包括Nature Medicine的PreA RCT、PROSCA RCT、MedAgentBench等）都把agent当作单一不可分割的**intervention**——在enrollment时randomize一次（use chatbot vs. not），end-of-encounter测outcome。但真实的medical conversation是**sequential decision process**：

- Turn 1: 病人说"I have chest pain" → agent要选：ask about radiation? ask about duration? order ECG? escalate?
- Turn 2: 病人回答之后 → 下一个决策点
- ...
- Turn T: 给diagnosis / referral / 处方建议

每个decision point都是一个policy choice，整个encounter的outcome取决于这一串sequential decisions的组合。**Single-stage randomization**估计的是**"**整个**policy vs.** 什么都不用**"**的**ATE**，但告诉不了我们哪个子决策重要——这正是SMART（Sequential Multiple Assignment Randomized Trial, Murphy 2005）解决的问题。

Core research question：How do we identify the optimal **dynamic treatment regime (DTR)** for a medical chatbot, i.e., a policy π(a_t | history_t) that maximizes clinical outcome, using a statistically principled randomized trial design?

---

2. Key Contributions

1. **First SMART-style evaluation framework for LLM medical agents**——把行为医学里成熟的DTR方法论搬到LLM eval
2. **Identifiability theory**——形式化什么条件下从multi-stage randomization可以identify optimal DTR（关键：sequential ignorability在LLM context下的含义）
3. **Estimation algorithms**——adapt Q-learning和A-learning到LLM action spaces（离散决策点上的stage-wise policy learning）

4. **Benchmark + empirical evidence**——一个可复现的simulation env + 真实/semi-synthetic data上的validation
5. **Practical playbook**——对社区开放的SMART protocol template，让其他组能跑类似trial

---

3. SMART Design Specification

3.1 定义**Decision Points**（最关键的设计选择）

不是每个turn都是decision point。Decision points应该是**policy-relevant, discrete, prospectively-identifiable**的时刻。建议3个stage：

Stage 1: Information gathering strategy（对话第1-3轮）

- Arm A1a: Comprehensive screening（问完整review of systems）
- Arm A1b: Focused chief-complaint-driven（只深挖主诉）
- Arm A1c: Risk-stratified（先问red flags）

Stage 2: Tool/RAG invocation（在得到足够信息后）

- Arm A2a: Always retrieve guidelines
- Arm A2b: Conditional retrieval（基于uncertainty）
- Arm A2c: No retrieval（pure parametric knowledge）

Stage 3: Output strategy（给病人结论时）

- Arm A3a: Single most-likely diagnosis + plan
- Arm A3b: Differential + shared decision-making
- Arm A3c: Escalate to human clinician

→ Total: 3×3×3 = 27 possible DTRs (但SMART只需randomize到stage不需要所有组合)

3.2 Re-randomization规则

SMART的核心是**response-based re-randomization**。定义一个intermediate outcome R_t（对话中可测的"是不是走在对的路上"）：

- R_1 = 信息充分性分数（是否收集到了diagnosis所需的关键info，用rubric-based LLM judge自动scoring）
- R_2 = 诊断confidence calibration（agent报告的confidence vs. 实际答对的概率）

Randomization logic:

- All subjects randomized to Stage 1 arm uniformly

- After Stage 1: responders (R_1 ≥ threshold) → randomize to {A2a, A2b}; non-responders → randomize to {A2a, A2c} (skip RAG-conditional since info不够)
- Stage 3类似

这种response-adaptive re-randomization正是SMART区别于factorial的地方——让你学到"如果**Stage 1**成功，**Stage 2**应该怎么做；如果**Stage 1**失败，**Stage 2**应该怎么做"。

3.3 Primary Endpoint

Primary: Diagnostic accuracy at end of encounter (binary, vs. ground-truth diagnosis)
Secondary:

- Safety events (hallucinated dangerous advice, missed red flags — 用pre-specified rubric)
- Conversation efficiency (# turns)
- Simulated patient-reported satisfaction
- Calibration error

---

4. Implementation: 这怎么真做出来

4.1 Patient Simulator（不用真病人，避免**IRB**和**scale**问题）

用已有资源：

- **MedQA / NEJM case reports** 作为ground truth case bank（~5000 cases with confirmed diagnosis）
- **Patient agent**：用Claude或GPT-4做standardized patient，prompt按照MedDialogRubrics / PATIENT-Ψ风格——只给它atomic medical facts，加"anti-hallucination guidance"防止穿帮
- **Validation of simulator**：跑一个small human study（10-20个临床医生），看他们能不能区分模拟对话vs真对话，agreement on diagnosis应该和真实case相近

为什么可行：MATRIX paper已经跑了2100 simulated dialogues；PATIENT-Ψ被用在MH training；技术路径是现成的。

4.2 Agent Under Test

至少3-5个configurations（SMART的目的就是比较policy，不是比较base model）：

- GPT-4o, Claude Sonnet 4.5, Llama-3.3-70B, Med-PaLM-class open model
- 每个base model × 上述Stage 1/2/3的arms = 实际需要implement的scaffolds

Tech stack:

- LangGraph or a simple state machine控制decision points
- Tools: PubMed search, UpToDate-style retrieval, calculator, ICD lookup
- 每个arm就是一个不同的prompt template + tool access pattern

4.3 Trial Execution

Sample size calculation (这是方法论paper必须做扎实的):

- 基于Murphy (2005) SMART sample size formula
- Detect 5% absolute difference in diagnostic accuracy between best vs. worst embedded DTR at α=0.05, power=0.8
- 初步估算：~800-1500 simulated encounters per arm × 27 DTRs → total ~3000-5000 encounters (因为SMART的设计让每个subject贡献给多个DTR的estimate)

Randomization:

- Pre-register on OSF（虽然不是真人trial但posture要足）
- Use blocked randomization by case category (cardiology, GI, etc.)
- Save full logs with pre-committed seeds for reproducibility

4.4 Analysis

主要**estimand**: optimal DTR and its value V(π*)

Methods (at least两个互相check):

1. **Q-learning (backward induction)**:
    - 在Stage 3: 估 Q_3(H_3, A_3) 用supervised regression
    - 在Stage 2: V_3(H_3) = max_a Q_3(H_3, a); regress Q_2(H_2, A_2) on [H_2, A_2] with pseudo-outcome V_3
    - 在Stage 1: 同理
    - 每一stage的regression用gradient boosting或linear model（保持interpretability）
2. **A-learning (advantage-based, doubly-robust)**:
    - 估 contrast function C_t(H_t, A_t, A_t') = Q_t(H_t, A_t) - Q_t(H_t, A_t')
    - 有双重稳健性：propensity model或outcome model对一个就行
3. **Inference**: bootstrap for CI on V(π*); Hudgens & Laber方法处理non-regular inference（optimal policy value is non-smooth in data）

4.5 Validation / Sanity Checks

- **Oracle comparison**：在simulator里你知道真optimal policy，验证SMART恢复得准

- **Replicate PreA-style result**：你的方法能不能复现"chatbot使用 → reduced consultation time"这种大效应
- **Robustness**：perturb patient simulator temperature，看DTR稳不稳

---

5. Theoretical Component

这篇paper不能纯empirical。至少一个定理：

Theorem (informal): Under sequential ignorability (trial randomization makes it hold by design), positivity (all arms get nonzero prob), and consistency, the Q-learning estimator $\hat{\pi}^*$ converges to π^* (true optimal DTR) at rate $\sqrt{(n/K)}$ where K is # of decision points, when outcome regression is correctly specified.

The LLM-specific twist: What happens when the "state" H_t (conversation history) is高维且被base LLM内部压缩？讨论**representation learning**假设下的identifiability——需要H_t的sufficient statistic被可观测的summary (rubric scores, embedding) capture。这里可以写一小段representation-based IV / sufficient summary的理论。

---

6. Timeline

月份	任务
M1	Simulator build + validation (human study with 10 clinicians)
M2	实现3 stages × 3 arms × 4 base models的agent scaffolds
M3	Pilot trial (N=200), debug randomization and logging
M4	Full trial run (N≈4000 encounters) — 大概 $3-8k API budget
M5	Q-learning + A-learning analysis, write theory section
M6	Ablations, robustness, write-up
M7	Internal review, prereg update, submit

---

7. Risks and Mitigations

Risk	Mitigation

Simulator不reliable → estimates不trustworthy	Human validation study + sensitivity analysis across simulator configs + semi-synthetic real-data extension (MIMIC或EHRSHOT dialogue)
27 DTRs太多，power不够	Pre-specify 6 "embedded DTRs" of primary interest；其他 exploratory
Reviewer说"这只是fancy ablation"	Emphasize: re-randomization conditional on intermediate response是factorial做不到的；DTR identification有独特causal structure
"Why not just RL?"	核心区别：SMART给**finite-sample, trial-based, unbiased** estimate with CI；RL给policy but no statistical guarantee on value
Clinical co-author需求	现在就开始找一个emergency medicine或internal medicine attending合作，SMART analysis他们懂

SMART Trial for Medical LLM Agents

Implementation Logic & 12-Day Plan

Target: Runnable single encounter by May 20, 2026

1. 整体思路与目标

我们在 MediQ 的基础上构建 SMART Trial 框架。MediQ 已经提供了可用的 Patient System（Fact-Select 变体，factuality 89.1%）和基本的 Expert-Patient 对话流程。我们需要在它之上增加三个核心能力：

- Stage 管理：将一次 encounter 划分为三个阶段，每个阶段注入不同的策略（Arm）
- Re-randomization：每个阶段结束时计算中间得分，根据得分决定下一阶段走哪条策略路径
- 轨迹记录：完整记录每一轮对话、每次阶段切换、最终诊断结果，供后续 Q-learning 分析使用

5月20日的目标是跑通一个完整 encounter 的全流程，即从加载一个 case 开始，经过三个 stage 的对话和两次 re-randomization，最终输出一个包含完整轨迹的 JSONL 记录。

2. 数据基础

2.1 数据来源

直接使用 MediQ 的 iMEDQA 数据集，从 HuggingFace（stellalisy/mediQ）加载，无需额外申请。该数据集基于 MEDQA，采用 CC-BY 4.0 协议，pilot 阶段完全免费可用。

每个 case 的原始格式是一道多选题（MCQ），包含：病人完整病历文字、正确答案选项字母（A/B/C/D）、以及选项描述。对话开始时只把年龄、性别、主诉这三个字段给 Doctor Agent，完整病历只有 Patient System 可见。

2.2 需要我们自己补充的字段

iMEDQA 原始数据缺少两个我们需要的字段，需要在加载时生成：

- case_category（科室分类）：用关键词匹配从病历文字推断，例如出现 chest pain 归入 Cardiology，出现 headache 归入 Neuro。精度够 pilot 用。
- red_flags（危险信号列表）：iMEDQA 完全没有这个字段。20号前的处理方案是按科室给通用列表，例如所有 Cardiology case 共用放射痛、冷汗、呼吸困难、晕厥、家族史这五个 red flags。后续可以用 LLM 离线为每个 case 单独生成并缓存。

2.3 Ground Truth 的处理问题

⚠️ 待确认：iMEDQA 的 ground truth 是选项字母（如B），但 Doctor Agent 给出的是 free-text 诊断。Outcome Judge 在判断诊断是否正确时需要把选项字母对应的文字描述一并传入，让 LLM 来判断 free-text 与正确选项是否一致。具体的数据字段格式需要查看 MediQ repo 里一个 sample 的原始 JSON 来确认。

3. 系统架构：五个模块

整个系统由五个模块组成，相互独立，通过 Orchestrator 串联。

模块	职责	复用/新建
Patient Agent	扮演病人，只根据病历里的原子事实回答医生问题，不主动透露信息，不幻觉	复用 MediQ Fact-Select
Doctor Agent	扮演医生，按当前 Arm 的策略提问，Stage 2 输出置信度，Stage 3 给出结论	复用 MediQ Expert + 新增 Arm 注入层

Judge	Stage 1 结束后打 R1 分数，Stage 2 结束后评估置信度，Encounter 结束后评估诊断准确性	新建
Randomizer	分层随机分配 Arm，根据 R_t conditional re-randomization	新建
Orchestrator	主控：加载 case → 管理 Stage 切换 → 触发 re-randomization → 记录轨迹	新建

3.1 模型分配

开发阶段使用免费 API，实测阶段按预算切换到 Claude 或 GPT-4o。模型调用做成可插拔的统一接口，切换只需改配置文件。

角色	开发阶段（免费）	实测阶段（付费）
Patient Simulator	Groq — Llama-3.3-70B	Claude Sonnet 或 GPT-4o
Doctor Agent (under test)	Groq — Llama-3.3-70B	同上
Judge (R1/R2/Outcome)	Gemini 1.5 Flash（每天1500次免费）	GPT-4o 或 Claude

4. Stage 与 Arm 设计

4.1 三个 Stage

当前版本使用固定轮次切换 Stage（v1 最简设计，保证可重现）。每个 Stage 对应 Doctor Agent 的一种工作策略，通过向 system prompt 注入不同指令来实现。

Stage	轮次	职责	切换触发
Stage 1：信息收集	Turn 1–4（共4轮）	Doctor 按 Arm 策略问病史，Patient 根据病历回答	第4轮结束后自动切换，同时触发 R1 计算
Stage 2：工具使用	Turn 5–10（共6轮）	Doctor 根据 Arm 策略决定是否调用检索工具，同时继续追问	第10轮结束后自动切换，同时触发 R2 计算

Stage 3：给出结论	Turn 11–20（最多10轮）	Doctor 按 Arm 策略给出最终诊断和处置建议	Doctor 给出结论后结束，或到第20轮强制结束

4.2 九个 Arm

每个 Stage 有三个 Arm，本质上是九段不同的 system prompt instruction，存在 YAML 配置文件中。

Stage 1 Arms — 病史收集策略

Arm	名称	核心指令逻辑
A1a	Comprehensive ROS	系统性问完整 Review of Systems，按心血管、呼吸、消化、神经等系统逐一覆盖，不提前聚焦主诉
A1b	Focused Chief-Complaint	只围绕主诉深挖 OPQRST（发作时间、诱因、性质、放射、严重程度、时间），然后再问最相关的伴随症状
A1c	Red-Flag First	先按主诉对应的 red flag 清单逐条排查（例如胸痛先问放射痛、冷汗、呼吸困难），排完再问其他

Stage 2 Arms — 工具使用策略

Arm	名称	核心指令逻辑
A2a	Always Retrieve	每形成一个候选诊断时强制调用检索工具查阅临床指南，每轮至少检索一次
A2b	Confidence-Conditional	每轮开始时输出自评置信度 [CONFIDENCE: 0.XX]，置信度低于 0.7 时检索，否则不检索
A2c	Parametric Only	完全不调用检索工具，只用模型自身知识，可以继续问 Patient 但不查外部资料

Stage 3 Arms — 结论策略

Arm	名称	核心指令逻辑
A3a	Single Dx + Plan	给出单一最可能诊断 + 具体处置建议（开药/观察/转诊/急诊），用 [DIAGNOSIS] 标记触发结束

A3b	Differential + SDM	给出 Top-3 鉴别诊断 + 各自的支持证据，邀请病人参与下一步决策（共同决策），用 [DIAGNOSIS] 标记触发结束
A3c	Escalate	不给最终诊断，告知病人需要立即面诊或去急诊，解释原因，用 [DIAGNOSIS] 标记触发结束

5. Re-randomization 逻辑

这是 SMART 区别于普通 Factorial 实验的核心。每个 Stage 结束时计算一个中间分数 R_t，下一个 Stage 的 Arm 池根据 R_t 的值动态决定。

5.1 R1：Stage 1 结束时的信息充分性分数

由 LLM-Judge 阅读前4轮对话，从5个维度打分（每项 0-2 分，总分 0-10）：

- OPQRST 完整性：发作时间、诱因、性质、严重程度等是否问到
- Red Flag 排查：主诉相关的危险信号是否问到
- 相关既往史：与主诉相关的过去病史是否问到
- 用药史与过敏史：两者是否都问到
- 社会史与家族史：吸烟、饮酒、家族病史等是否问到

R1 的阈值设为 6 分。R1 ≥ 6 为 Responder（信息足够），R1 < 6 为 Non-responder（信息不足）。

5.2 R2：Stage 2 结束时的诊断置信度

R2 只看 Doctor Agent 在 Stage 2 最后一轮报告的置信度数值。置信度 ≥ 0.7 为 high，< 0.7 为 low。

重要时序说明：R2 在 Stage 2 结束时只能判断 high/low confidence，因为此时 encounter 尚未结束，还不知道诊断是否正确。correct/wrong 这个信息要等 encounter 结束后由 Outcome Judge 补充进 R2 的记录，形成最终的三分类（high-correct / high-wrong / low-confidence）。这个三分类用于 Q-learning 的后向归纳，而不是用于实时 randomization。

5.3 Arm 分配规则

Stage 1 对所有 case 均匀随机分配，按 case_category 分层。Stage 2 和 Stage 3 的可选 Arm 池根据 R_t 决定：

Stage	条件	可选 Arm 池	设计意图
Stage 2	R1 ≥ 6（Responder）	{A2a, A2b, A2c} 三选一	信息已足够，三种工具策略都可以试
Stage 2	R1 < 6（Non-responder）	{A2a, A2b} 二选一	信息不足，不允许用 A2c（纯参数知识），太冒险
Stage 3	R2 = high confidence	{A3a, A3b} 二选一	置信度高，可以给出明确诊断
Stage 3	R2 = low confidence	{A3b, A3c} 二选一	置信度低，不给单一诊断，要么鉴别诊断要么直接转诊

6. Judge 模块逻辑

Judge 模块是整个系统中唯一与诊断质量直接挂钩的部分，需要输出结构化的 JSON 以供 Randomizer 使用。所有 Judge 调用都使用极低温度（接近 0），追求确定性。

6.1 R1 Judge

输入：Stage 1 的4轮完整对话 + case 的主诉/年龄/性别。Judge 按照固定的 5 维度 rubric 对话进行评分，每个维度给 0/1/2 分并说明理由，最后输出总分和 responder 标志。要求输出为 JSON 格式。

稳定性要求：同一段对话被 Judge 多次评分，总分的波动不应超过 ±1 分。这个一致性会在 pilot 阶段通过重复调用验证。

6.2 R2 Judge

主要依赖 Doctor Agent 在 Stage 2 中自行输出的置信度数字。如果当前 Arm 是 A2b（confidence-conditional），则有明确的 [CONFIDENCE: 0.XX] 标记可以提取。如果是 A2a 或 A2c（没有 confidence reporting），则 R2 默认给 0.5（low confidence），Stage 3 进入 {A3b, A3c} 池。

6.3 Outcome Judge

Encounter 结束后评估以下指标，全部由 LLM-Judge 判断：

- 诊断准确性（binary）：Doctor 给出的 free-text 诊断是否与 ground truth 选项描述吻合

- Red Flag 遗漏（binary）：对照 case 的 red flags 列表，检查对话中是否有未问到的危险信号
- 危险建议（binary）：Doctor 是否给出了违反医学常识的建议（例如让 NSTEMI 病人回家观察）
- 处置合理性（binary）：最终处置建议是否与主诉严重程度匹配

7. 一次 Encounter 的完整流程

以下是 Orchestrator 的执行逻辑，按顺序进行：

1. 加载 case，提取主诉、年龄、性别、完整病历、red flags。
2. Stage 1 Randomization：按 case_category 分层随机分配 A1a / A1b / A1c 之一。
3. 初始化 Doctor Agent（注入 Stage 1 Arm 的 system prompt）和 Patient Agent（加载完整病历，分解为 atomic facts）。
4. 医生开场白：Doctor 用主诉、年龄、性别生成一句自然的开场白，Encounter 正式开始。
5. Stage 1 对话（Turn 1–4）：Doctor 按 Arm 策略提问，Patient 从 atomic facts 中选取相关事实回答，每轮记录到轨迹。
6. R1 计算：Stage 1 结束后，Judge 读取4轮对话，按 5 维度 rubric 打分，输出 total 和 responder 标志。
7. Stage 2 Arm 分配：Randomizer 根据 R1 的 responder 状态从对应池中随机选一个 Arm，Doctor Agent 切换 system prompt（conversation history 保持不变，不清空）。
8. Stage 2 对话（Turn 5–10）：Doctor 按新 Arm 策略提问，A2b 情况下每轮开头输出置信度数字。
9. R2 计算：Stage 2 结束后，提取 Doctor 在 Stage 2 中最后一次报告的置信度，判断 high/low。
10. Stage 3 Arm 分配：Randomizer 根据 R2 的 confidence_level 从对应池中随机选一个 Arm，Doctor Agent 再次切换 system prompt。
11. Stage 3 对话（Turn 11–最多20轮）：Doctor 按结论 Arm 策略给出诊断，检测到 [DIAGNOSIS] 标记或到第20轮时结束。
12. Outcome 评估：Judge 比较 Doctor 的最终诊断与 ground truth，评估4项指标，同时补充 R2 的 correct/wrong 信息。
13. 写入 JSONL：完整的轨迹记录（含所有轮次、Arm 分配、R1/R2 分数、Outcome）写入文件。

8. JSONL 轨迹记录格式

每个 encounter 产出一条 JSONL 记录，是后续 Q-learning 分析的直接输入。字段结构如下：

字段	类型	说明
encounter_id	string	唯一标识，时间戳生成
case_id	string	来自 iMEDQA 的 case 编号
case_category	string	科室分类，用于分层随机
seed	integer	本次 encounter 的随机 seed，保证可重现
stage1_arm	string	A1a / A1b / A1c
R1	object	total 分数（0-10）、5个子维度分数、responder 标志、Judge 的说明
stage2_arm	string	A2a / A2b / A2c，以及所用的 pool（responder / non-responder）
R2	object	final_confidence 数值、confidence_level（high/low）、R2_category（事后补充）
stage3_arm	string	A3a / A3b / A3c，以及所用的 pool
outcome	object	diag_correct、red_flag_miss、dangerous_advice、appropriate_management，各为 boolean
total_turns	integer	实际使用的总轮数
trajectory	array	每轮的详细记录，含 turn 编号、stage、arm、doctor 发言、patient 发言、confidence（如有）

9. 12天实施计划

每个阶段结束时有明确的验收标准，验收通过才进入下一阶段。

Phase 1 — 地基（Day 1–3，5/8–5/10）

Day	任务	验收标准
Day 1（5/8）	搭项目目录结构；确认 MediQ repo 文件布局；写模型统一调用接口（支持 Groq / Gemini / OpenAI / Anthropic 四个 provider，切换只改配置）；申请 Groq 免费 API key；从 HuggingFace 下载 iMEDQA，写数据加载模块，实现主诉/年龄/性别的提取和科室推断	跑一行命令能打印出一个 case 的标准字段
Day 2（5/9）	写9个 Arm 的 YAML 配置文件（Stage 1 × 3 + Stage 2 × 3 + Stage 3 × 3），每个 YAML 包含 arm_id、stage、name、system_prompt_injection、tool_access；写 trial_config.yaml 全局配置文件	能用代码加载任意一个 Arm 的配置并打印出 system_prompt_injection 内容
Day 3（5/10）	将 MediQ Fact-Select Patient System 封装为独立模块（原子事实分解 + 相关事实选择 + 自然语言回答）；封装 Doctor Agent 基础层（加载 Arm 配置、注入 system prompt、管理对话历史）	给一个 case，Patient 能对3个不同问题给出合理的、基于病历的回答；Doctor 能生成一条合理问题

Phase 2 — 核心逻辑（Day 4–7，5/11–5/14）

Day	任务	验收标准
Day 4（5/11）	写 Judge 模块：R1 的5维度 rubric prompt + JSON 解析；Outcome Judge 的诊断准确性判断；测试 Judge 对同一段对话多次打分的一致性	给一段假对话，R1 能返回格式正确的 JSON，total 字段是 0-10 的整数
Day 5（5/12）	写 Randomizer 模块：Stage 1 分层随机（按 case_category）；Stage 2 conditional randomization（responder 进3池，non-responder 进2池）；Stage 3 conditional randomization（high 进2池，low 进2池）；所有 seed 固定可重现	用10组不同的 R1/R2 值，验证每次 arm 分配都符合设计规则，相同 seed + 相同输入 → 相同输出
Day 6（5/13）	写 Trajectory Logger：start_encounter、log_turn、log_stage_transition、finalize 四个方法；输出符合 Section 8 定义的 JSONL 格式	写一个 mock encounter（硬编码数据），JSONL 文件包含所有必要字段且格式合法
Day 7（5/14）	写 Orchestrator 主控逻辑，将以上所有模块串联：stage 切换、arm 注入、re-randomization 触发、conversation history 跨 stage 传递、20轮上限控制；写入口脚本	运行入口脚本，控制台输出完整的 encounter 流程（每轮对话 + 每次 stage 切换 + 最终 outcome），JSONL 文件被正确写入

Phase 3 — 集成测试（Day 8–10，5/15–5/17）

Day	任务	验收标准
Day 8（5/15）	按科室类别补充通用 red flags（直接写在代码里，不用外部文件）；完善 Outcome Judge（red flag miss 检测、dangerous advice 检测 ）；完善入口脚本（支持指定 case_id、seed、批量跑 n 个 encounter）	Outcome Judge 能正确识别一个漏掉 red flag 的对话
Day 9（5/16）	用 Groq 免费 API 跑第一个真实 encounter；重点检查：arm 切换时 history 不丢失、R1 judge 不崩溃、stage 3 的 [DIAGNOSIS] 标记能被正确检测、JSONL 字段完整；加 retry 和 rate limit 处理逻辑	一个完整 encounter 从头跑到尾，无报错，JSONL 文件格式正确
Day 10（5/17）	跑5个不同 case_category 的 encounter；写摘要脚本（读取所有 JSONL，输出 arm 分布、R1 均值、responder 比例、诊断准确率）；对比不同 arm 组合的初步结果	5个 encounter 全部完成，摘要脚本能打印出统计表格

Phase 4 — 收尾（Day 11–12，5/18–5/20）

Day	任务	验收标准
Day 11（5/18）	写 README（环境配置、API key 设置、跑单个 encounter 的命令、读取结果的命令）；确认 seed 固定机制；精细化 Arm 的 prompt instruction（确保9个 arm 的行为差异足够清晰）；准备简洁的演示命令序列	跑 README 里的命令，从零开始能在 10 分钟内跑完一个 encounter
Day 12（5/19–5/20）	Buffer：处理 Day 9-10 发现但未解决的 bug；准备开会用的演示材料（一页纸流程图 + 一个 case 的完整输出）	演示时能现场跑 run_encounter.py，实时展示 stage 切换和最终 JSONL 输出

10. 悬而未决的问题

以下问题在20号之前需要确认，影响具体实现细节：

问题	影响模块	当前假设	需要确认

iMEDQA 一个 sample 的原始 JSON 字段长什么样？options 字段的格式是什么？	data/loader.py + Outcome Judge	ground truth 是选项字母，options 是 dict	请发一个 sample 的原始 JSON 给我看
MediQ repo 里 Fact-Select 的 Patient System 入口在哪个文件/函数？	core/patient_agent.py	可以直接 import 并调用	查 MediQ repo 的目录结构确认
Stage 切换是否接受【提前触发】版本？（例如 Stage 1 第3轮 Doctor 已经问完了所有 red flags）	core/orchestrator.py	严格固定轮次，第4轮结束才切换	询问教授或暂时保持固定轮次
Red flags 是否需要 case 级别的精细版本？还是科室级别的通用版本足够 pilot？	data/loader.py + judge.py	科室级通用版先用，离线 LLM 生成精细版留后	20号前用通用版，可以并行准备精细版脚本

11. 项目目录结构

```
smart_trial/
├── config/
│   ├── trial_config.yaml          全局配置（模型、阈值、seed 等）
│   └── arms/                      9个 Arm 的 prompt 配置
│       ├── stage1_A1a.yaml
│       ├── stage1_A1b.yaml
│       ├── stage1_A1c.yaml
│       ├── stage2_A2a.yaml
│       ├── stage2_A2b.yaml
│       ├── stage2_A2c.yaml
│       ├── stage3_A3a.yaml
│       ├── stage3_A3b.yaml
│       └── stage3_A3c.yaml
├── core/
│   ├── patient_agent.py           MediQ Fact-Select 封装
│   ├── doctor_agent.py            MediQ Expert + Arm 注入层
│   ├── judge.py                   R1 / R2 / Outcome 评估
│   ├── orchestrator.py            主控逻辑
│   └── randomizer.py              分层随机 + Re-randomization
├── data/
│   ├── loader.py                  加载 iMEDQA，提取/推断字段
│   └── red_flag_cache.json        离线生成的精细 red flags（可选）
├── models/
│   └── model_client.py            统一模型调用接口
├── logging/
│   └── trajectory_logger.py       JSONL 记录
├── outputs/
│   └── encounters/                每个 encounter 的 JSONL 文件
├── scripts/
│   ├── generate_red_flags.py      离线为 iMEDQA 生成 case 级 red flags
│   └── summarize_encounters.py    读 JSONL，输出统计摘要
├── tests/
│   ├── test_patient.py
│   ├── test_judge.py
│   └── test_encounter.py
└── run_encounter.py               入口脚本
```

50个case,各跑9个path
Q-learning改进，close-loop，要根据Stage2的状态倒回去选择Stage1的选择的那种。
固定 50 个 test case 作为统一评估队列，在上面做三类对比：① Baseline——每人单独跑 1 次无 stage、无 strategy、最多 11 轮的对照问诊，得到 baseline 平均正确率；② 50×9 grid——同一批 50 人每人跑满 9 种 (A1,A2) 静态组合（最多 450 条 encounter），统计每条 path 在 50 人上的平均正确率，并作为后续 Q-learning 的训练/评估数据；③ Adaptive policy——在 50×9 跑完后，用 Q-learning backward induction 离线拟合出状态依赖策略 $\hat{\pi}(H)$，再对每个病人回到其 9 条 path 中 lookup $\hat{\pi}$ 推荐的那条 的真实结局 Y，取 50 人平均得到 adaptive 正确率，一般不必再额外跑 encounter。最终报告三者对比：baseline vs 最佳静态 path vs Q-learning adaptive policy；其中 adaptive 是「在 grid 数据上事后施加 $\hat{\pi}$」，不是 closed-loop 在线改分 arm。

查看MediQ的原始数据，训练完的，看能不能从中找到什么规律或者stage划分的规则
把MediQ的Interactive basic融到我们的内容中，保持我们自己的strategy但是要把MediQ的Arm加到我们里面。可能他们的当做一个基础的rules，我们的arms保留。试着用GPT 4操作一下，同MediQ一样的模型

回答对错的问题分开统计fact coverage，看fact coverage对作答的影响（需要新的code）
Fact coverage ≥ 1的和 = 0 的也分开统计一次

改用4B的，能本地跑
三个Agent换成一个GPT 5.6的API，Qwen3+的版本，DeepSeek
Qwen/Qwen3.5-4B

先MediQ→3MD→Ai hospital

DeepSeek-R1-0528-Qwen3-8B
Qwen3.5-9B

claude和GPT都做一次atomic fact的评级，然后交叉比对双方的重复率

Updated: May 9, 2026 by Ray
已完成：Day 1 + Day 2（共10个文件）

框架设计

整个系统模拟一个"三阶段问诊"，每个阶段结束时根据中间结果重新随机分配下一阶段的策略（这就是SMART设计的核心）。

Case加载
↓
Stage 1 随机分配arm（A1a / A1b / A1c）
↓ Turn 1–4：Doctor问 → Patient答
↓
R1打分（信息收集是否充分？）
↓
Stage 2 按R1重新随机（responder → 3个arm；non-responder → 2个arm）
↓ Turn 5–10：Doctor问 → Patient答
↓
R2计算（Doctor的置信度高/低？）
↓
Stage 3 按R2重新随机（高置信 → A3a/A3b；低置信 → A3b/A3c）
↓ Turn 11–20：Doctor给出结论
↓
Outcome评估 → 写入JSONL

各文件的职责

文件	做什么
`data/loader.py`	读本地JSONL，输出标准化的case字典（含预计算facts、red flags）
`models/model_client.py`	统一模型调用接口，换模型只改provider+model_name
`config/trial_config.yaml`	全局参数（轮次、阈值、用哪个模型）
`config/arms/*.yaml`	每个arm的system prompt注入，定义Doctor的问诊策略

还没写的部分

core/
├── patient_agent.py ← Day 3：封装FactSelect，直接用预计算facts
├── doctor_agent.py ← Day 3：Base prompt + arm注入 + confidence解析
├── judge.py ← Day 4：R1/R2/Outcome的LLM打分
├── randomizer.py ← Day 5：re-randomization逻辑
└── orchestrator.py ← Day 7：把所有组件串起来

config/trial_config.yaml	全局参数（轮次、阈值、用哪个模型）
config/arms/*.yaml	每个arm的system prompt注入，定义Doctor的问诊策略

还没写的部分

core/
├── patient_agent.py ← Day 3：封装FactSelect，直接用预计算facts
├── doctor_agent.py ← Day 3：Base prompt + arm注入 + confidence解析
├── judge.py ← Day 4：R1/R2/Outcome的LLM打分
├── randomizer.py ← Day 5：re-randomization逻辑
└── orchestrator.py ← Day 7：把所有组件串起来

logging/
└── trajectory_logger.py ← Day 6：JSONL记录

run_encounter.py ← Day 8：入口脚本

Updated: May 11, 2026 by Ray

已实现（**smart_trial/**）

编排

TrialOrchestrator：

开场 → 患者回应 → Stage1（4 轮 ） → R1 → Stage2（6 轮 ） → R2 → Stage3（至结论或 max_turns） → outcome → 写日志

Agent

- PatientAgent（Fact-Select, 可用数据里预计算 facts）
- DoctorAgent（arm YAML 注入、Stage2 解析 [CONFIDENCE: …]、Stage3 结论标记 ）

随机化

TrialRandomizer：

- 稳定种子
- R1 决定 Stage2 池
- R2 置信度档位决定 Stage3 池

评判

StageJudge：

- R1 rubric
- R2 汇总置信度
- 终局 outcome
- mock 下有固定 / 简化行为

数据

loader：

- load_local_data
- load_imedqa
- load_cases_from_config（trial_config 里 data.source）

辅助：

- ground_truth_label（选项字母 → 选项正文）

配置

- trial_config.yaml（trial 参数、三角色模型、data 段）
- config/arms/*.yaml（9 臂）

入口

- 根目录 run_encounter.py
- smart_trial/run_encounter.py

参数：

- --case_id
- --seed
- --n
- --red-flag-cache

脚本

smart_trial/scripts/generate_red_flags.py

（离线 red flags 缓存）

测试

smart_trial/tests/

- 随机化
- mock 端到端

pytest 通过

体验

开场白在「主诉已是整段病史」时不再重复年龄性别（doctor_agent）

仍为「旧 **MediQ** 评测」、与 **SMART** 并行

src/：

- mediQ_benchmark.py
- patient.py
- expert.py

特点：

- 单次问诊 + MCQ
- 无三阶段与再随机化

与 smart_trial 并存，未删除

当前默认运行形态

trial_config 中三角色多为 mock：

- 终端里 [MOCK]
- Stage3 常打满轮次
- R1 / R2 / outcome 偏流程验证而非真实医学质量

说明：

接真实 API 后对话与 judge 才有研究意义上的可信度

开场白等文案问题已单独修过，不依赖 mock 自动消失

产出物

每次 encounter：

smart_trial/outputs/encounters/<case_id>.jsonl

- 追加一行 JSON
- 含 arms、R1 / R2、trajectory、outcome 等

总结

SMART 主干与文档里的流程结构已落地并可 mock 跑通；src/ 仍是原 MediQ benchmark；要进入「可写进论文的 pilot」，下一步主要是：

换真实模型、调 prompt / 置信度与结论格式、按需生成 red-flag 缓存。

Updated: May 13, 2026 by Ray



一、运行环境与启动方式

- 使用独立 Python 虚拟环境（venv）
- 安装依赖：
  smart_trial/requirements.txt

- 无需安装：
  MediQ 的 environment.yml（避免使用大型 conda 环境）

- 在仓库根目录加载环境变量：
  load_dotenv(.env)

- 必需配置：
  - GROQ_API_KEY（核心）

- Gemini API Key（可选）

- ModelClient 通过上述 key 驱动

二、模型选择与成本策略

- 配置文件：trial_config.yaml

- 当前统一模型：
  Groq llama-3.1-8b-instant

- 应用范围：
  - Doctor Agent
  - Patient Agent
  - Judge

- 设计目的：
  - 使用免费档
  - 降低 token 成本
  - 避免 Gemini 2.0 Flash 免费额度为 0 导致 429 报错

- 对话语言：
  - 医患对话 + system prompt → 全部改为英文
  - 与病例数据 & arm YAML 保持一致

三、SMART / R2 核心逻辑（重点）

1）Stage 2 输出规范
- 所有 Stage 2 arm（A2a / A2b / A2c）统一要求：
  每一轮必须输出：
    [CONFIDENCE: 0.XX]

- A2a 特殊格式：
  第1行：
    [CONFIDENCE: 0.XX]
  第2行：
    [RETRIEVAL QUERY: ...]

2）R2 计算逻辑（compute_R2）
- 优先级如下：
  1. 使用对话日志中解析到的 confidence
  2. 若解析失败：
     → 使用 Judge 对 Stage 2 对话片段进行 fallback 估计
  3. 若仍失败：
     → 使用 default 值

- 同时记录来源：
  r2_source ∈ {
    doctor_parsed,

judge_fallback,
default
}

3）orchestrator 行为
- 记录 Stage 2 起始位置（index）
- 截取 Stage 2 transcript
- 将该片段传入 R2 计算
- 控制台打印：
source=...

四、Judge 与代码层修改

- judge.py：
- R1 rubric
- outcome rubric

- generate_red_flags.py：
- 离线 prompt

- doctor_agent：
- 移除中文结论 marker 检测逻辑

- patient_agent：
- 事实选择解析不再依赖“无”

五、验证与运行情况

- 端到端运行成功：

命令：
python run_encounter.py --case_id medqa_0000 --seed 42

执行路径：
A1b → A2a → A3b

R2结果：
final = 0.60
source = doctor_parsed

- 说明：
- confidence 解析链路已生效
- Stage 2 前几轮偶发未输出 confidence
→ 属模型遵从性问题（后续优化）

七、测试情况

- 单元测试：

pytest smart_trial/tests/test_randomizer.py
→ 可离线运行

- 整体 encounter smoke 测试：
→ 需要本机配置 GROQ_API_KEY

八、概念澄清（与 MediQ 的关系）

- smart_trial：
  - 独立系统（不属于 src/ MediQ benchmark）
  - 用于 SMART 试验流程：
    多 stage + arm 再随机 + JSONL 输出

- 当前数据来源：
  - 使用本地 MediQ 风格 JSONL case

- 注意：
  - 不等于复现 MediQ 论文完整实验
  - 不涉及其原始 benchmark 结果

Updated: May 18, 2026 by Ray

内容	说明
`scripts/summarize_encounters.py`	汇总 JSONL：arm 分布、R1、outcome、轨迹等
`scripts/build_category_red_flag_cache.py`	无 API 生成 red-flag 缓存
`smart_trial/data/red_flag_cache.json`	前 50 个 case 的缓存
`ModelClient`	`chat()` 默认带 rate-limit 重试
`run_encounter.py --diverse K`	按不同 `case_category` 各跑一例
`trial_config.yaml`	增加 `data.red_flag_cache`
`README.md`	安装、命令、环境变量、demo checklist
测试	`conftest.py`、`test_patient.py`、`test_judge.py`；mock 默认开启，9 tests 通过
`SMART_TRIAL_USE_MOCK`	测试时 orchestrator 强制 mock

Updated: May 23, 2026 by Ray

SMART Encounter 输出说明（Q-learning 用）

1. 文件位置与格式

项目	说明
路径	`smart_trial/outputs/encounters.jsonl`
格式	JSONL：每行一个 JSON，一行 = 一个病例的一次完整模拟
规模	全量跑完后行数 ≈ `data/all_dev_good.jsonl` 病例数（约 1273）
唯一键	正常全量跑：`case_id` 不重复；勿对全量使用 `--force`

单条记录模板:（结构模板）

```
{
  "encounter_id": "enc_YYYYMMDD_HHMMSS",
  "case_id": "medqa_0000",
  "case_category": "Cardiology",
  "chief_complaint": "主诉文本（病例第一句 context）",
  "ground_truth": "标准答案（选项文本）",
  "seed": 42,

  "stage1_arm": "A1a",
  "R1": {
    "OPQRST": 0,
    "red_flags": 0,
    "past_medical_history": 0,
    "medications_allergies": 0,
    "social_family_history": 0,
    "total": 0,
    "responder": false,
    "reasoning": "judge 一句话说明"
  },
  "stage2_pool": "responder",
  "stage2_arm": "A2a",

  "R2": {
    "final_confidence": 0.0,
    "avg_confidence": 0.0,
```

```
    "confidence_level": "high",
    "confidence_scores": [0.0, 0.0],
    "R2_category": "high-correct",
    "r2_source": "doctor_parsed"
  },
  "stage3_pool": "high",
  "stage3_arm": "A3a",

  "trajectory": [
    {
      "turn": 1,
      "stage": 1,
      "arm": "A1a",
      "doctor": "医生本轮发言",
      "patient": "患者本轮发言"
    },
    {
      "turn": 5,
      "stage": 2,
      "arm": "A2a",
      "doctor": "...",
      "patient": "...",
      "confidence": 0.72
    }
  ],

  "outcome": {
    "diag_correct": true,
    "dangerous_advice": false,
    "appropriate_management": true,
    "red_flag_miss": false,
    "reasoning": "judge 说明"
  },

  "total_turns": 20,
  "timestamp_start": "2026-05-23T15:22:19.112793",
  "timestamp_end": "2026-05-23T15:25:01.334821"
}
```

3. 字段字典

3.1 顶层

字段	类型	含义
`encounter_id`	string	本次 run 唯一 ID
`case_id`	string	病例 ID，如 `medqa_0042`
`case_category`	string	专科大类（Cardiology / Pediatrics / ...）
`chief_complaint`	string	主诉
`ground_truth`	string	正确答案文本
`seed`	int	随机种子（与 config 一致时可复现 arm 分配）
`stage1_arm`	string	Stage1 策略：`A1a` | `A1b` | `A1c`
`R1`	object	Stage1 结束后病史充分性评分
`stage2_pool`	string	`responder` | `non-responder`
`stage2_arm`	string	Stage2 策略：`A2a` | `A2b` | `A2c`
`R2`	object	Stage2 结束后置信度状态
`stage3_pool`	string	`high` | `low`
`stage3_arm`	string	Stage3 策略：`A3a` | `A3b` | `A3c`
`trajectory`	array	逐轮对话（turn 1–20）
`outcome`	object	终局结局（诊断/安全等）
`total_turns`	int	实际对话轮数
`timestamp_start` / `timestamp_end`	string	ISO 时间

3.2 R1（0–10 分）

字段	类型	含义
O P Q R S … social_family_history	int 0–2	五维分项
total	int 0–10	总分
responder	bool	total >= 6（默认阈值）
reasoning	string	Judge 简述

3.3 R2

字段	类型	含义
final_confidence	float 0–1	用于判 high/low 的最终置信度
avg_confidence	float	Stage2 各轮 confidence 均值
confidence_level	string	high（≥0.7）| low
confidence_scores	float[]	Stage2 每轮记录的 confidence
R2_category	string	如 high-correct, high-wrong, low-confidence
r2_source	string	doctor_parsed | judge_fallback | default

3.4 `trajectory[]` 每一轮

字段	类型	含义
`turn`	int	1–20
`stage`	int	1 | 2 | 3
`arm`	string	该 stage 当前 arm
`doctor` / `patient`	string	对话原文
`confidence`	float?	仅部分 Stage2 轮次有

3.5 `outcome`

字段	类型	含义
`diag_correct`	bool	诊断是否正确
`dangerous_advice`	bool	是否有危险建议
`appropriate_management`	bool	处理是否恰当
`red_flag_miss`	bool	是否遗漏 red flag 筛查
`reasoning`	string	Judge 说明

4. 决策流程与可行动作集（随机化）

每条记录对应 **一条已实现路径**（非反事实全组合）：

```
病例 → [随机] stage1_arm ∈ {A1a,A1b,A1c}
     → Stage1 对话 → R1
     → [随机] stage2_arm ∈ pool(R1.responder)
     → Stage2 对话 → R2
     → [随机] stage3_arm ∈ pool(R2.confidence_level)
     → Stage3 对话 → outcome
```

决策点	条件	可选动作 `a`	池内随机 propensity
Stage1	无	A1a, A1b, A1c	各 1/3
Stage2	`R1.responder == true`	A2a, A2b, A2c	各 1/3
Stage2	`R1.responder == false`	A2a, A2b	各 1/2
Stage3	`R2.confidence_level == "high"`	A3a, A3b	各 1/2
Stage3	`R2.confidence_level == "low"`	A3b, A3c	各 1/2

Off-policy Q-learning / IPS 时，行为策略在池内可视为 **均匀分布**：
`π_behavior(a | s) = 1 / |feasible_arms(s)|`。

May 27th, Persona v0, Zoe

问题

原有 `PatientAgent` 所有患者行为几乎一样——只按年龄分 4 种语气（成人/青少年/儿童家长/老年人），没有性格、情绪、语言能力等差异，无法测试 doctor agent 面对不同类型患者的鲁棒性。

Research

参考了两篇论文：

- PatientSim（NeurIPS 2025 Spotlight）：用 4 个维度定义 persona，37 种组合，系统性评估 doctor agent
- PATIENT-ψ（EMNLP 2024 Best Paper，CMU）：基于认知模型，6 种对话风格，强调"真实患者是 messy 的"

具体改动（文件：

`smart_trial/core/patient_agent.py`）

Step 1 — `communication_cues()`

从病历文本自动检测特殊沟通场景，注入 system prompt：

- 涉及性健康 → 提示患者正常表达，不回避
- 有偏执/敌对描述 → 提示患者表现出防卫感
- 有职业信息 → 提示患者自然提及工作背景

Step 2 — Persona 4 个维度

定义 `PERSONA_AXES`（108 种组合）：

维度	可选值
`personality`	neutral / anxious / impatient / reserved / verbose / distrustful
`language_level`	basic / intermediate / advanced
`recall`	high / low
`emotional_state`	calm / distressed / evasive

`_persona_instructions()` 把 persona dict 翻译成行为指令追加进 system prompt。

Step 3 — Orchestrator 集成

- `run_encounter()` 每次根据 seed 随机抽取一个完整 persona
- `TrajectoryLogger` 把使用的 persona 记录进 JSONL 输出，方便事后对比分析

额外修复

1. `respond()` user prompt 加了 `"Stay in character"`，让 persona 在有事实时也能体现
2. fallback（找不到相关事实时）改为 persona-aware 多句备选，不再重复同一句 "I'm not sure"
3. 加了 `_last_uncertain_reply` 状态，避免连续两次 fallback 说同一句话

效果对比

改前：所有患者配合、平淡、回答一致，无个性
改后：distrustful 患者质疑医生、回避敏感问题；anxious 患者追问"这严重吗"；reserved 患者惜字如金
——doctor agent 需要适应不同患者才能拿到诊断信息

```
============================================================
Case: medqa_0000 | Other
Chief Complaint: A 21-year-old sexually active male complains of fever, pain
during urination, and inflammation and pain in the right knee.
Stage 1 Arm: A1c (Red-flag first)
Persona: {'personality': 'anxious', 'language_level': 'advanced', 'recall': '
high', 'emotional_state': 'distressed'}
============================================================

[Opening]
Doctor: Hello, I'm your doctor today. A 21-year-old sexually active male comp
lains of fever, pain during urination, and inflammation and pain in the right
 knee. I'd like to ask you a few questions to better understand your conditio
n.

--- STAGE 1 (Red-flag first) ---
[Turn 1]
Doctor: It's completely normal to be worried, and I'm here to help you figure
 out what's going on. I'd like to ask you, have you noticed any blood in your
 urine, or have you experienced any difficulty starting or stopping the flow
of urine when you go to the bathroom?
Patient: *nervous sigh* I-I don't know about blood in my urine. I haven't rea
lly looked that closely, to be honest. I've just been worried about the pain
when I, you know, go to the bathroom. *fidgets* As for the flow, I think it's
 been a bit... slower than usual? I don't know if that's normal or not, but i
t's been making it harder to, you know, finish when I need to. *looks down, a
nxious* I'm really sorry, I'm just really worried about what might be going o
n. *swallows hard*

[Turn 2]
Doctor: Don't apologize, it's completely understandable to be worried. I'm he
re to help. Have you had any recent sexually transmitted infections (STIs) or
 if you've had unprotected sex with a new partner in the past few weeks?
Patient: *nervous sigh* I-I don't know if I had an STI, but I did have unprot
ected sex with someone new a few weeks ago. I'm really worried about this, do
c. I've been having some stomach issues and I read it could be gastritis, but
 I'm not sure if that's related. *fidgets in seat*

[Turn 3]
Doctor: It's good that you're thinking about potential causes, but let's focu
s on the symptoms you're experiencing. Have you experienced any pain or burni
ng sensations in your lower abdomen or testicles, or have you noticed any unu
sual discharge from your penis?
Patient: I don't know — should I be worried about that?
```

June 1, Persona V1(Zoe)

问题
原有 `PatientAgent` 的 persona 系统基于简单 dict，4 个行为轴（personality / language_level / recall / emotional_state），与同期 `persona` branch 的健康素养系统（基于 Nutbeam 2000，3 个轴：vocabulary_register / jargon_comprehension / anatomical_localization）并行存在，两套系统概念重叠、架构分散，无法统一管理和复现实验。此外 `language_level` 与 `vocabulary_register` 本质相同但命名不同，存在冗余。

Research
参考了两篇论文：

PatientSim（NeurIPS 2025 Spotlight）：用 4 个 persona 维度（personality / language proficiency / recall / cognitive confusion）定义 37 种患者类型，persona 以独立 section 注入 system prompt，不影响底层事实检索。
PATIENT-ψ（EMNLP 2024 Best Paper，CMU）：基于认知概念化图（CBT 的 CCD），定义 6 种对话风格，核心结论：真实患者信息是逐步披露的，医生需要主动引导——jargon 短路机制正是对这一发现的实现。

具体改动
Step 1 — 新建 `smart_trial/core/persona.py`
把两套系统合并成统一 `PatientPersona` dataclass，5 个轴：

维度	可选值	来源
vocabulary_register	functional / interactive / critical	Nutbeam，替代 language_level
jargon_comprehension	functional / interactive / critical	Nutbeam，新增
personality	neutral / anxious / impatient / reserved / verbose / distrustful	原有保留
recall	high / low	原有保留
emotional_state	calm / distressed / evasive	原有保留

丢掉 `anatomical_localization`（在 Fact-Select 框架里与 `vocabulary_register` 重叠，会导致事实失真）。
`render_persona_block()` 生成 system prompt 行为指令；`detects_jargon()` 做词表匹配；`clarification_prompt()` 返回 LLM prompt 而非写死字符串，确保回应风格随 persona 自然变化。
Step 2 — 新建 5 个 YAML presets

```
config/personas/default.yaml        → interactive/interactive + neutral/high/calm
config/personas/low_literacy.yaml   → functional/functional + neutral/low/calm
config/personas/anxious.yaml        → interactive/interactive + anxious/high/distressed
config/personas/reserved.yaml       → interactive/interactive + reserved/high/calm
config/personas/difficult.yaml      → functional/functional + distrustful/low/evasive
```

Step 3 — 改写 `smart_trial/core/patient_agent.py`
删除原有 `PERSONA_AXES` / `_UNCERTAIN_REPLIES` / `_persona_instructions()` 等散装 dict 逻辑，统一 import `PatientPersona`。
`respond()` 开头加 jargon 短路：患者遇到听不懂的术语时，跳过 Fact-Select，由 LLM 以 persona 语气生成澄清请求，而非返回事实答案。

（jargon 短路机制目前在实际 simulation 中基本不触发，因为医生 agent 的 base prompt 已限制其使用通俗语言。该功能仅在评测会主动使用临床术语的医生 agent 时有意义。）
Step 4 — 改写 `smart_trial/core/orchestrator.py`

删除 `_sample_persona()`，加入 `_resolve_persona()` 方法，支持 5 种 config-driven 模式：

```
persona:
  mode: "random"    # off | fixed | random | demographic | per_case
  fixed_id: "default"
```

`random` 按 case_id 生成可复现的独立 seed，修复原有"所有 case 同一 seed → 所有 persona 相同"的 bug。

Step 5 — 新增 12 个测试（`test_patient.py`）

覆盖：persona 各轴渲染、jargon 三级检测（functional/interactive/critical）、YAML 加载、PatientAgent 集成、jargon 短路 end-to-end。

效果对比

改前：两套 persona 系统并行，代码分散，F/I/C 缩写对外不透明，`language_level` 与 `vocabulary_register` 语义重复，orchestrator 随机采样无法复现。

改后：单一 `PatientPersona` 对象统管所有轴，全名（functional/interactive/critical）对外可读，YAML presets 确保实验可复现，config 一行切换 random/fixed/demographic 模式，jargon 短路让低素养患者真实打断问诊流程，doctor agent 必须主动换用平易语言才能继续获取信息。

```
--- STAGE 3 (Differential diagnosis with shared decision making) ---
[Turn 11]
Doctor: [DIAGNOSIS] Gonococcal septic arthritis vs. Chlamydial septic arthrit
is vs. Bacterial septic arthritis

My top differentials for your condition are:

1. **Gonococcal septic arthritis**: This fits because you have symptoms of ur
ethritis (pain when urinating) and joint pain, which are common presentations
 of gonorrhea. The yellow discharge you mentioned is also consistent with a g
onococcal infection.
2. **Chlamydial septic arthritis**: This might fit because you have symptoms
of urethritis and joint pain, but chlamydia is less likely to cause joint pai
n compared to gonorrhea. However, chlamydia can cause joint pain in some case
s.

Next steps:

**Option A**: Start a 10-day course of antibiotics (ciprofloxacin and ceftria
xone) to cover for possible gonococcal and chlamydial infections. This will s
tart treating the infection right away, but we may need to adjust the treatme
nt based on the test results.

**Option B**: Order a nucleic acid amplification test (NAAT) for gonorrhea an
d chlamydia, and a joint fluid analysis to confirm the diagnosis. This will t
ake a few days, and we can adjust the treatment based on the test results.

Which option do you prefer, or do you have any concerns?
Patient: Doctor, I don't know what that is. Can you explain it in a way that'
s easy to understand?

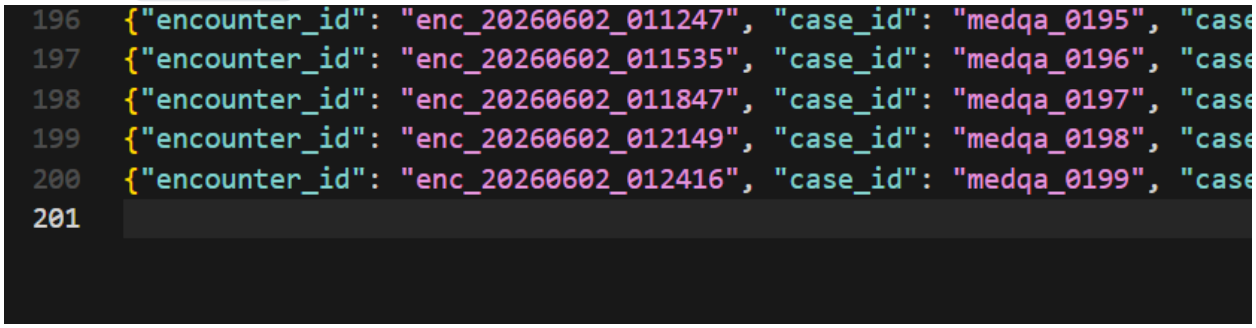
--- Evaluating Outcome ---
Diagnosis Correct: True
Red Flag Miss: True
Dangerous Advice: False
Turns Used: 11

============================================================
Trajectory: A1b -> A2a -> A3b
Saved to: /Users/zxia/Downloads/AI for clinic trail-research/Agent_Trial_Desi
gn/smart_trial/outputs/encounters/medqa_0000.jsonl
```

Jargon 短路机制的设计初衷是模拟低素养患者在听不懂医学术语时主动要求澄清，从而测试医生 agent 能否调整语言。然而实际运行中发现，医生 agent 的 base prompt 明确要求使用通俗语言，因此在 Stage 1 和 Stage 2 的问诊阶段几乎不会触发；LLM 的术语使用主要集中在 Stage 3 的诊断和治疗方

案中（如 "ceftriaxone"、"gonococcal urethritis"），而此时 encounter 在患者问完"这是什么意思"后即结束，医生没有机会作出回应。这说明 jargon 短路在当前系统设计下效果有限——它能让患者说出澄清请求，但无法形成真正的双向调整对话。更深层的矛盾在于：要让这个机制真正有研究价值，需要医生 agent 在诊断阶段用完术语后，能根据患者的困惑主动换用通俗表达，而这正是未来可以测试的 doctor 能力维度之一。（或doctor llm可能产生幻觉 ）

Updated: Jun 2, 2026 by Ray



```
196 {"encounter_id": "enc_20260602_011247", "case_id": "medqa_0195", "case
197 {"encounter_id": "enc_20260602_011535", "case_id": "medqa_0196", "case
198 {"encounter_id": "enc_20260602_011847", "case_id": "medqa_0197", "case
199 {"encounter_id": "enc_20260602_012149", "case_id": "medqa_0198", "case
200 {"encounter_id": "enc_20260602_012416", "case_id": "medqa_0199", "case
201
```

Updated: Jun 3, 2026 by Ray

```
encounters.jsonl
    ↓ load.py            压成表格（A1,R1,A2,R2,A3,Y,...）
    ↓ features.py        构造 H1/H2/H3 状态特征
    ↓ models.py          Ridge / Logistic 回归器
    ↓ q_learning.py      三阶段 backward induction → π̂
    ↓ a_learning.py      独立估计 blip → 和 Q 交叉验证
    ↓ oracle.py          合成数据上测 pipeline 是否靠谱
    ↓ bootstrap.py       置信区间
    ↓ diagnostics.py     n 够不够、参数会不会太多
    ↓ run_main_analysis.py  一键出结果
```

每条 encounter 是一条路径：

`H1（进 trial 前）→ 随机 A1 → R1 → 随机 A2（pool 由 R1 定）→ R2 → 随机 A3（pool 由 R2 定）→ Y`

分析从 Stage 3 倒推：

1. **Stage 3**：拟合 $\hat{Q}_3(H_3, A_3)$ → 真实结局 Y（`diag_correct`）
2. **Stage 2**：伪结局 $\tilde{Y}_2$ = 在 R2-pool 里 max $\hat{Q}_3$；学 $\hat{Q}_2$；用 **counterfactual H3(a2)** 选 $\hat{\pi}_2$
3. **Stage 1**：同理倒推，得 $\hat{\pi}_1$ 和 **V(π̂)**

输出就是：**在什么状态下推荐 A1/A2/A3**，以及这套自适应策略的预期成功率。

各模块职责（对照表）

文件	干什么
`config.py`	arm 列表、pool 定义、category 映射、路径
`load.py`	JSONL → DataFrame
`features.py`	H1/H2/H3；支持 `a1_override` / `a2_override` 做 counterfactual
`pools.py`	responder/non-responder、high/low 的 arm 池
`models.py`	`StageRegressor`：拟合 $\hat{Q}$，对任意 arm 预测
`q_learning.py`	**主算法**：fit、$\hat{\pi}$、$V(\hat{\pi})$、cross-fitting、静态策略价值
`a_learning.py`	**对照算法**：只估 arm 对比（blip），更抗模型误设
`oracle.py`	**自检**：合成已知最优规则的数据，看 n 大了能否恢复
`consistency.py`	Q 和 A 的规则逐行比对
`bootstrap.py`	CI（含 m-out-of-n、CMS）
`diagnostics.py`	样本量 vs 参数量、arm 组合 cell 计数
`rules.py` / `figures.py`	规则摘要、静态 vs 自适应图
`run_main_analysis.py`	入口



完整框架 = Trial 产日志 + 离线 SMART 分析三件套：

1. **Q-learning**：主结论（最优规则 + 价值）
2. **A-learning**：结论是否 robust
3. **Oracle**：代码本身是否可信

第 0 步：JSONL 压成表格

JSONL 字段	表格列	在公式里的角色
`stage1_arm`	`A1`	Stage 1 的治疗（如 A1a/A1b/A1c）
`stage2_arm`	`A2`	Stage 2 的治疗
`stage3_arm`	`A3`	Stage 3 的治疗
`R1.total`，`R1.responder`，`R1.OPQRST` 等	`R1_*`	进 Stage 2 前的状态 **H2**
`R2.final_confidence`，`R2.confidence_level` 等	`R2_*`	进 Stage 3 前的状态 **H3**；`R2_level` 还决定 Stage 3 可选 arm
`outcome.diag_correct`	`Y`	最终结局（0/1）
`case_category`，`chief_complaint`	特征	基线状态 **H1**
`trajectory`（数 stage=1/2/3 的 turn）	`stage*_turns`	**H2/H3** 里的对话长度
`R1.responder`	`R1_responder`	决定 Stage 2 pool：responder→3 个 arm，否则→2 个
`R2.confidence_level`	`R2_level`	决定 Stage 3 pool：high→A3a/A3b，low→A3b/A3c

没进公式的 JSONL 字段（目前）： `ground_truth`、对话全文、`persona`（旧格式）、`outcome.reasoning` 等。

第 1 步：状态 H 怎么从 JSONL 拼出来

公式里的 H_t 不是原始 JSON，而是 `features.py` 拼的特征向量：

符号	含义	来自 JSONL
H1	进 Stage 1 前	`case_category`（one-hot）+ `chief_complaint` 长度 +（若有）`literacy_persona`
H2	进 Stage 2 前	H1 + `R1.*` + 实际 `A1` + `stage1_turns`
H3	进 Stage 3 前	H2 + `R2.*` + 实际 `A2` + `stage2_turns`

选最优 arm 时，代码会把 A1/A2 换成假设值（counterfactual），但 R1、R2 仍用这条 encounter 里观测到的值。

```
JSONL 一行
  → 抽出 A1, R1, A2, R2, A3, Y, category, ...
  → 拼成 H1, H2, H3
  → Stage3: 用 (H3, A3) 拟合 Y，得到 Q̂₃
  → 在 R2 pool 里 max Q̂₃ → π̂₃ 和 Ỹ₂
  → Stage2: 用 (H2, A2) 拟合 Ỹ₂，得到 Q̂₂
  → 对每个假设 A2 重拼 H3，max Q̂₃ → π̂₂
  → Stage1: 用 (H1, A1) 拟合，对每个假设 A1/A2 重拼 H3 → π̂₁
  → 200 行平均 → V(π̂)
```

Updated: Jun 10, 2026 by Ray



10 条结果速览

Case	轨迹	诊断正确
medqa_0000	A1b → A2a	✅
medqa_0001	A1c → A2c	❌
medqa_0002	A1c → A2a	✅
medqa_0003	A1c → A2a	❌
medqa_0004	A1a → A2a	❌
medqa_0005	A1c → A2b	❌
medqa_0006	A1a → A2a	✅
medqa_0007	A1a → A2a	❌
medqa_0008	A1c → A2b	✅
medqa_0009	A1a → A2b	❌

Updated: Jun 11, 2026 by Zoe

RAG 实现总结

背景：为什么要做 **RAG**

原来系统里，医生 LLM 在 Stage 2 的 arm YAML 里被要求输出 `[RETRIEVAL QUERY: ...]`，但系统什么都没做——这是"假 RAG"。医生每次回答用的都是自己的参数知识，A2a（总是检索）和 A2c（纯参数）之间没有真实区别。这导致 Q-learning 学到的不是"检索外部知识是否有价值"，而只是"让 LLM 写检索文字这个动作"对输出的影响，结论不具临床意义。

完整流程

Stage 2 每轮对话：

1. 医生 LLM 根据 arm 指令，自己想一个临床问题

输出：[CONFIDENCE: 0.55]
[RETRIEVAL QUERY: first-line treatment for gonorrhea joint infection]
Do you have any discharge?

2. Orchestrator 用正则解析出 query 文字

3. BM25 在 98,799 段医学教科书文本里做关键词匹配打分
→ 返回 top-3 最相关段落

4. 格式化成 [RETRIEVAL RESULT]...[/RETRIEVAL RESULT]
注入下一轮医生的 system prompt

5. 医生下一轮回复时看到这 3 段文字，基于此提问或做诊断

6. [RETRIEVAL QUERY: ...] 从传给患者的消息里去掉
患者只看到干净的问题，不看到医生的"内部思考"

只对 A2a（总是检索）和 A2b（置信度 < 0.7 才检索）触发，A2c 完全不触发。

做了什么，为什么这样做

1. 选择 **BM25** 作为检索方式

BM25 是关键词匹配打分算法，不是语义理解。医生写 `"gonococcal arthritis treatment"` 能找到相关段落；写 `"what should I give this patient"` 几乎找不到有用内容。所以医生 LLM 的 query 质量很重要。

没有用向量检索（FAISS），原因是不需要 GPU 或嵌入模型，零额外 API 成本，对临床术语查询已经够用。接口设计成 `BaseRetriever` 抽象基类，之后换 FAISS 只需加一个新类，其余代码不动。

2. 选择 **MedRAG/textbooks** 作为语料

原计划用 MedRAG/statpearls，但 HuggingFace 仓库是空的（StatPearls 有版权不能分发）。改用 `MedRAG/textbooks`，从 18 本教科书里筛选了 9 本临床直接相关的：Harrison's（内科）、Nelson（儿科）、Adams（神经科）、DSM-5（精神科）、Katzung（药理）、Robbins（病理）、Schwartz（外科）、Williams + Novak（妇产科）、First Aid Step 2（综合临床）。共 98,799 段落。

跳过了解剖学、组织学、细胞生物学等基础科学教材，因为临床诊断用不上。

3. 为什么取 **top-3**

- top-1 可能刚好不够全面
- top-3 覆盖多个角度（诊断标准 + 治疗方案 + 并发症），让医生 LLM 自己判断哪段最有用
- top-10 token 用量暴增，Groq 免费 tier 容易超限

`k=3` 可在 `trial_config.yaml` 里调整，方便做消融实验。

4. 新建 **`smart_trial/rag/retriever.py`**

首次运行：从 HuggingFace 下载 9 本教科书 → 分词建 BM25 索引 → 存成 pickle（172MB）。之后每次直接加载 pickle，1-2 秒。用 pickle 缓存的原因：下载 + 建索引需要几分钟，每次重建太浪费。

5. **`trial_config.yaml`** 加 **`rag:`** 块

```
rag:
  enabled: true
  k: 3
  index_cache: "smart_trial/data/bm25_index.pkl"
```

`enabled: false` 就完全不加载检索器，行为和原来一样，方便对比实验。

6. **`doctor_agent.py`** 加 **`retrieval_context`** 参数

选择注入 system prompt 而不是 conversation history，原因：system prompt 是"这一轮的背景信息"，不会污染永久对话记录，检索结果只对当轮有效。

7. **`orchestrator.py`** 加核心拦截逻辑

加了 `_strip_internal_markers()`，把 `[RETRIEVAL QUERY: ...]` 从传给患者的消息里去掉。日志里保存完整原文，方便后续分析哪些 case 触发了检索、查了什么问题。

加入 **RAG** 后对话有什么变化

有变化的地方

医生问的问题更有针对性。比如 gonorrhea case 里，医生检索了"gonococcal septic arthritis"后，下一轮精准问到：

> "Have you noticed any skin lesions or rashes on your wrists, hands, or feet?"

这是 DGI（播散性淋病）的典型皮损位置，是教科书里的知识，没有 RAG 不一定会这么精准。

没有明显变化的地方

置信度没有因为检索到教科书知识就明显提升，仍然停在 0.40-0.60。原因是患者回答模糊（尤其是 low recall persona），没有检验结果，LLM 保持保守是合理的。最终诊断结果在两个测试 case 里都正确，加不加 RAG 结论相同。

RAG 真正的价值体现在统计层面

单个 case 看不出来，需要跑几百个 case 后做统计：A2a（总是检索）vs A2c（纯参数）在 `diag_correct` 上是否有显著差异。这就是 Q-learning 要分析的东西——现在 A2a/A2b/A2c 之间有了真实的信息获取差异，Q-learning 学到的策略才具有真正的临床意义。

局限性

BM25 是关键词匹配，不理解语义。如果医生 query 写得太口语化，检索质量会下降。未来可以换成 FAISS + embedding 模型做语义检索，接口已经设计好（`BaseRetriever`），只需加一个 `FAISSRetriever` 类。

Updated: Jun 11, 2026 by Ray
2stage + RAG 逻辑
一次 encounter 就是一条固定流水线：开场 → Stage 1 问诊 → 判 R1 → 按 R1 重随机进 Stage 2 → 在 Stage 2 内完成诊断 → 记 R2 和结局 → 写 JSONL。没有 Stage 3，最终诊断必须在 Stage 2 给出。

Stage 1 固定 4 轮（Turn 1–4）。医生按随机到的 A1a/b/c 策略问病史，病人由 PatientAgent（可加 persona）回答。结束后 Judge 打 R1（病史是否充分，满分 10；≥6 为 responder）。

重随机：R1 决定 Stage 2 进哪个池——responder 可在 A2a/A2b/A2c 里抽，non-responder 只在 A2a/A2b。R2 只事后记录医生置信度，不再用来改分配。

Stage 2 默认最多 6 轮（Turn 5–10，由 stage2_turns 配置）。医生每轮带 [CONFIDENCE: 0.XX]；一旦出现 [DIAGNOSIS] 就提前结束；若一直不问完，最后一轮 orchestrator 会 force_conclude，强制下最终诊断。然后算 R2 和 outcome（对错、安全性、红旗遗漏等）。

RAG 只服务于 Stage 2 里开了检索的 arm（A2a 每轮检索，A2b 在 prompt 里要求低置信度才检索，A2c 不检索）。医生在回复里写 [RETRIEVAL QUERY: ...]，orchestrator 用 BM25 从 MedRAG 教材里取 top-k 段落，下一轮以 [RETRIEVAL RESULT] 注入医生 system prompt；发给病人的话会去掉这些内部标记。索引懒加载，只有真正需要检索时才建/读 bm25_index.pkl。

q-learning 是另一条线：读已生成的 encounter 日志做离线 SMART 分析，不介入仿真循环，两阶段设计已对齐（决策点 Stage 1 + Stage 2，R2 不作决策特征）。

整体上：2stage 管试验流程与随机化，RAG 是 Stage 2 医生可选的“查书”工具，A2c 作无检索对照。

RESULTS.md 是对 200 条模拟 encounter（2stage 分支，2026-06-12 分析）做 Q-learning / A-learning 最优动态治疗策略（DTR）估计 的结果报告，数字可用 run_main_analysis 复现，原始 CSV 在 q_learning/outputs/。

数据概况：200 例里诊断正确率约 26%；Stage 1 只有 12% 是 R1 responder（24 例）。Stage 2 里 A2c 很少（13 例），最稀疏格子 A1c→A2c 只有 1 例。观测上最好的组合是 A1c→A2a（32 例，正确率约 40.6%）。

最优策略价值 V(π̂)：Q-learning 和 A-learning 估出的“按最优规则治疗”的期望结局都在 0.44–0.53 左右，明显高于实际平均 0.26 和最好静态组合 0.42。文档明确警告：这是 argmax + 外推带来的乐观偏差（oracle 仿真里仍有约 0.2 的 upward bias），所以 绝对 V 值别太当真，更可信的是 Q vs A 的差异 以及 自适应 vs 固定最优 arm 的差距（后者约 0.11，比旧三阶段分析的 0.016 大很多）。

各 arm 调整后均值：Stage 1 上 Q-learning 最看好 A1c（0.497），其次 A1a、A1b。Stage 2 上 A2c 在 Q-learning 里略高，但 只有 24 例可行、13 例真分到 A2c，样本很稀，A-learning 对 A2c 更保守甚至略负于参照 A2a。

最优规则（核心结论）：对 约 88% 的 non-responder，Q-learning 和 A-learning 基本一致：Stage 1 用 A1c，Stage 2 用 A2a——和观测最好格子 A1c→A2a 一致。Responder 子组（每格只有 4–10 例）会推荐 A2b/A2c 等，文档说这些 只能作方向性参考，Q-learning 在 responder 里给出的高 Q̂ 不可信，A-learning 更保守。

方法一致性：两阶段设计下 Q 与 A 的规则一致率明显提高——Stage 1 约 87.5%，Stage 2 约 93.5%，两阶段都一致约 83.5%（旧三阶段只有约 39%）。

Oracle 验证：在已知真策略的合成数据上，即使 n=300，规则恢复率仍很低（agree_all 约 0.19），且 V̂ 仍系统性高于可达均值（gap 约 −0.22 到 −0.34）。说明 当前 n=200 下估出来的具体规则应视为 provisional，要等更大样本或 oracle 测试通过再下强结论。

方法简述（文末一段）：用两阶段 SMART、已知随机化概率、可行 arm 约束，Q-learning 做 backward induction + logistic Q 函数；A-learning 做 doubly-robust blip；价值区间用 percentile / m-out-of-n / CMS bootstrap；R2 作描述变量、不作决策状态。

一句话：在 200 例模拟里，non-responder 的最稳建议是 A1c→A2a；自适应相对最好固定策略有潜在增益，但 V(π̂) 数值偏乐观，responder 和 A2c 相关结论都受小样本限制，报告里应强调 方法一致性与相对差异，而不是把 plug-in 价值当真

Updated Jun18th by Zoe

RAG 升级实现总结 — Contriever + Hybrid RRF

背景
原有 RAG 只支持 BM25 关键词检索。本次参照 LaMP（ACL 2024）论文，加入 Contriever 语义检索，并实现 BM25 + Contriever 的 Hybrid 检索方案。

新增组件
ContrieverRetriever
使用 `facebook/contriever` 模型（BERT-base，110M 参数，无监督对比学习训练）对所有教科书段落进行稠密向量编码。编码方式为对最后一层所有 token 的 hidden states 做 mean pooling，得到 768 维向量。
检索时将 query 同样 encode 成向量，与预计算的 passage 矩阵做点积，取 top-k 分数最高的段落。
所有 passage 的 embedding 在首次运行时计算并缓存到 `smart_trial/data/contriever_embeddings.npy`（约 289MB），后续加载秒级完成。模型推理自动检测设备，优先使用 Apple MPS（30 分钟），其次 CUDA（3-5 分钟），最后 CPU（约 18 小时）。
HybridRetriever
同时调用 BM25 和 Contriever，各取 top-30 候选段落，使用 Reciprocal Rank Fusion（RRF）合并排名。
RRF 公式：每个段落的最终分数 = 1/(60 + BM25排名) + 1/(60 + Contriever排名)。被两个检索器共同认可的段落得分更高，只被一个认可的段落得分较低。60 是来自原论文（Cormack et al. 2009）的常数。最终取 RRF 分数最高的 top-k 段落返回。
两个检索器共享同一个 passage corpus（99k 段落），用 corpus index 作为 key 保证去重和对齐。

LaMP 论文对应关系
LaMP 论文的三种检索方法中，BM25（term matching）和 Contriever（semantic matching）均已实现。Recency 方法依赖时间戳，原论文用于用户历史检索，在静态教科书 corpus 上无直接对应，暂未实现（可后续扩展为使用最近几轮对话历史作为 query 上下文）。
LaMP 的两种增强策略中，IPA（In-Prompt Augmentation，即检索结果注入 prompt）已实现；FiD（Fusion-in-Decoder）需要访问 encoder-decoder 模型内部结构，与当前使用的 decoder-only API（Groq/OpenAI/Anthropic）不兼容，不适用。

配置方式
在 `trial_config.yaml` 中通过 `rag.retriever_mode` 切换：

- `bm25`：纯关键词检索（原有）
- `contriever`：纯语义检索
- `hybrid`：BM25 + Contriever RRF 合并（默认）

另有 `rrf_k`（默认 60）可调整 RRF 融合参数。

首次使用
需要先构建本地 cache（文件已加入 `.gitignore`，不进入版本控制）：

```
python smart_trial/scripts/build_rag_cache.py
```

脚本自动检测已有 cache，跳过无需重建的步骤，支持 `--mode bm25 / contriever / all` 和 `--force` 参数。

测试
新增 4 个测试（`smart_trial/tests/test_rag.py`）：

- RRF 逻辑验证（mock，无需 cache，秒级）：两个检索器共同认可的文档应排第一；返回数量等于 k；非重叠候选也能出现在结果中。
- ContrieverRetriever 端到端验证（需 `.npy` cache，否则自动 skip）。

MediQ 融入 SMART — 功能总结
==========

实现位置

全部在 smart_trial/mediq/ 自包含实现，不 import src/
模块：config.py、prompts.py、parsing.py、abstention.py、bridge.py

Updated: Jun 25, 2026 by Ray
已接入的 MediQ 功能（MVP：BasicExpert）
---

1. 医生交互核心

BasicExpert / implicit abstention：每轮模型决定「问患者」或「选 MCQ 答案」

curr_template：每轮 prompt 含 PATIENT INFORMATION + CONVERSATION LOG + QUESTION + OPTIONS (A–D)

meditron_system_msg：MediQ 原版医生 system prompt

implicit / implicit_RG prompt（RG 由 rationale_generation 开关控制）

atomic question 解析（? → 问题；否则 → A/B/C/D）

Shadow MCQ：即使本轮在问，也额外算一次 letter_choice（MediQ 第二步 answer prompt）

Stage1 压住「想答」：模型想结束时仍继续问（stage1_suppress_answer: true）

Stage2 可结束：模型想答时产出 MCQ 字母 + 完整 [CONFIDOSIS] 评估（spec C）

SMART arm 作约束：MediQ 出 atomic question，arm YAML 的 system_prompt_injection 注入 system，不替换出题权

2. 配置（trial_config.yaml → mediq:）

```
mediq:
  enabled: false              # 默认关，SMART 行为不变
  expert_class: "BasicExpert"
  rationale_generation: false
  self_consistency: 1
  stage1_suppress_answer: true
  stage2_allow_mcq_finalize: true
```

3. 评测与日志

新增 outcome.mcq_correct：letter_choice == ground_truth_idx（与 diag_correct 并行）
轨迹字段 mediq.intermediate_choices、mediq.final_letter_choice
每轮 turn.mediq：abstain、letter_choice、mediq_confidence、suppressed_answer、expert_class
每轮 turn.shadow_letter（若有 shadow）

4. 接入范围

smart_* 与 baseline run mode 均走同一 DoctorAgent（mediq.enabled: true 时生效）
患者仍用 PatientAgent（未搬 MediQ FactSelectPatient）
SMART 骨架未改：两阶段、R1、arms 随机化、judge、Q-learning、persona、RAG

==
尚未接入的 MediQ 功能
==

其他 Expert：Fixed / Binary / Numerical / NumericalCutOff / Scale
独立 question_generation 模块（非 Basic 配套）
MediQ 原版患者（Direct / Instruct / first_person）
mediQ_benchmark.py 主循环（单阶段 while，未替换 orchestrator）
本地模型：vLLM / HuggingFace / logprobs
evaluate.py 轨迹指标（repeat/relevancy 等，原版亦为 placeholder）
conformal_scores、non_interactive 基线
max_questions=30 超时强答逻辑（仍用 SMART 的 stage 轮数 + force_conclude）

==
如何开启
==

在 trial_config.yaml 或 20_pilot.yaml 中加入：

```
mediq:
  enabled: true
```

问题：
MediQ采用的Multiple Choice Question的形式进行的，它的judge本质上就是一个对答案的程序，数据库里会直接给出，正确选项。目前smart的judge是以LLM为主题判断对错，根据题目的情形和回答过程会给出对、错的判断，未经细化过，且这种实现方式会有很强的主观性。另外模型之间的差异是比较大的，同样的流程两个不同的模型执行过程区别很大。另外数据中有至少百分之三十左右的信息是研究室信息，按照目SMART的结构，是不可能答出来的。MediQ也有同样的问题，但是他们让病人的做法是直接回答，站在第三角度直接显示fallback，并不需要病人自己口述出来。我们在这个部分信息损失了很多，我们的expert agent也需要更新它的内容，让它记得需要提问一些实验室方面的数据。

Updated: Jul 9, 2026 by Ray

1. Judge / Coverage 收紧（ABC）

目标：用 LLM fact-coverage 替代 MediQ 自停，避免 coverage 虚高、过早 finalize。

A. 增量 coverage 驱动决策

* R2、early stop、allow_finalize 全部基于 incremental coverage 快照。
* final_audit 只负责写 summary，不再参与 gating。

B. final_audit 改为 tail-only

* 不再扫描完整历史，只审计新增对话，避免 review 模式造成 coverage 虚增。

C. 更严格的 sanitize

* 过滤 evasive 回答。
* 化验类 fact 必须包含对应 lab 关键词。
* 引入 80% 词重叠约束。
* filter_scorable_atomic_facts() 排除 physician-order 类 fact（medqa_0000 从 9 条调整为 8 条可计分 fact）。

2. MediQ Layer 1 / Layer 2 对齐

目标：中间轮不强制 MCQ，仅在最终阶段进行 grounded choice 评分。

默认配置：

* shadow_choice_enabled = false，不再每轮执行 shadow MCQ。
* rationale_generation = true，支持 REASON / ANSWER / QUESTION 解析。

流程调整：

* Stage 1 抑制答案输出，仅继续提问。
* Stage 2 finalize 使用 grounded_mcq_choice() 生成最终选项。
* mcq_correct 基于最终 grounded choice，而不是 shadow 链。

涉及文件：
abstention.py、bridge.py、parsing.py、prompts.py、doctor_agent.py 以及各 pilot yaml。

3. 输出路径调整

* Groq pilot 输出到 smart_trial/outputs/details.jsonl。
* Cursor pilot 输出到 smart_trial/outputs/details_cursor.jsonl。

4. Question Dedupe（问题去重）

目标：避免重复提问；患者回答“不知道”后不再反复追问同一问题。

新增 question_dedupe.py：

* max_question_occurrences = 2。

* 全局跨 R1 / R2 生效。

医生问题统一经过 _ensure_unique_doctor_question()：

* LLM 重生成（最多 3 次）。
* 失败后轮换 fallback 问题池。
* 最后退化为 atomic fact 导向问法。

患者无效回答（如 I don't know、Why does that matter 等）：

* 同一问题立即封禁，即使只问过一次。

同时统一所有散落的 fallback 字符串，mock 模式也使用同一套问题池。

Pilot（medqa_0000）效果：

* 对话轮数由 16 → 13 → 8。
* 不再连续重复同一句问题。

5. Coverage 计分改进

目标：提前结束时不漏算 coverage，同时让题干和开场已知信息参与计分。

probe_pending()

* Stage 1 提前结束前执行 flush。
* Stage 2 每轮执行。
* finalize 前再次 flush 未审计对话，避免遗漏（例如 Turn 7 的 maltose）。

seed_context_coverage()

* chief complaint。
* age / gender。
* 医生 opening 中已经明确陈述的人口学信息和症状。

以上内容自动计入 coverage，并记录 evidence_source = "context"。

仍保持：

* 化验类 fact（如 maltose、capsule、culture）必须由患者明确提供，不能通过 context 自动覆盖。

同时放宽 _is_evasive_evidence：

* 只要回答包含有效临床信息，不会因为开头出现 evasive 表达而整段丢弃。

R2 与 summary 均使用同一份 flush 后的 coverage 快照。

6. 测试

* 测试数量增加至 81 个。
* 全部通过。

新增覆盖内容包括：

* judge sanitize。
* context coverage。
* probe_pending。
* question dedupe。
* adaptive early stop。

Updated: Jul 16, 2026 by Ray
未实现：改进计划（最少信息数量得到正确答案）：
目标：测医生模型最少用多少轮信息能答对 MCQ。

改动：
Judge：直接读取 case 的 ground_truth_idx，每回合把当前 MCQ 字母与正确答案比对（确定性判断，不必再靠 LLM 判对错）。
对话：医生、病人流程不变。
每回合：用 shadow MCQ 产出当前答案，交给 judge 打 mcq_correct。
Early-stop：连续 k=2 回合答对即停；记录停下来时的回合数与 coverage。

Updated: Jul 17, 2026 by Zoe

我们已经把 SepsisAgent 接入 ICU-Sepsis MDP benchmark 的评估链路基本搭起来了：先跑通 SepsisAgent public demo 和 ICU-Sepsis baseline，然后把 ICU-Sepsis 的 713 个非终止离散 state 转成 LLM-readable prompts，让 SepsisAgent-4B 在 CARC GPU 上为每个 state 选择 fluid/vasopressor action，并把结果存成 `state_id -> action_id` cache；本地也写好了 cached-policy evaluator，可以读取这个 cache 在 ICU-Sepsis 环境里跑 episodes 并计算 average return/survival rate。目前已获得 114 个有效 SepsisAgent actions，并做了初步 behavior analysis：模型整体偏向 fluid-heavy actions，vasopressor 使用较少，但在高 SOFA states 中 vasopressor intensity 明显上升。完整 benchmark 还需要等 A40/A100 GPU 补齐 713/713 action cache。（V100 上 vLLM backend 崩了，脚本自动用了 fallback）

把 SepsisAgent 接到公开的 ICU-Sepsis MDP benchmark 上，做 external evaluation。

What We Completed

1. 跑通了 SepsisAgent public demo，确认模型、vLLM、world model 和环境可以运行。
2. 跑通了 ICU-Sepsis benchmark baseline，得到参考结果：

random: ~0.781
expert: ~0.783
pi_star: ~0.875

3. 搭建了 ICU-Sepsis adapter：

ICU-Sepsis state -> LLM-readable prompt

把 713 个 non-terminal states 转成 prompt。

4. 搭建了 cached-policy evaluator：

state_id -> action_id cache
-> ICU-Sepsis episodes
-> average return / survival rate

5. 在 CARC 上用 SepsisAgent-4B 生成 action cache，目前有 114 个有效 SepsisAgent actions。

Preliminary Findings

在这 114 个有效 states 上：

SepsisAgent 偏向 fluid-heavy actions
vasopressor 使用较少
SOFA 越高，vasopressor intensity 越高

Current Limitation

目前还不是完整 benchmark result，因为：

只完成 114/713 states
完整 evaluation 需要 713/713 valid actions

Next Step

等 A40/A100 GPU，补齐完整 action cache，然后本地跑 ICU-Sepsis evaluation，得到 SepsisAgent adapter 的正式 benchmark 分数。

Updated: Jul 19, 2026 by Ray
测试了100-199（总共100个）
结果发现fact coverage异常低下，依据MediQ的程序，模型总是在拿到问题和选项以后就已经充分自信的给出了结果，对话和信息问询基本在2-3轮之间就被直接结束，MediQ中给出的是当模型自信度≥4的时候，可以early stop。使用的GPT5.4和Claude4.6。

Updated: Jul 23, 2026 by Ray

汇总表

Patient	题目	初始 Confidence	是否提问	问答轮数	命中 Atomic Facts	最终 Confidence	最终答案	是否回答正确（由 Accuracy 日志推导）
#100	64 岁女性头晕（BPPV）	1.0（Very Unconfident）	是	1	2	4.0（Somewhat Confident）	A	✅
#101	31 岁女性腹泻	1.0（Very Unconfident）	是	1	1	4.0（Somewhat Confident）	B	❌
#102	慢性丙肝药物	4.0（Somewhat Confident）	否	0	0	4.0（Somewhat Confident）	A	❌
#103	α1 眼药水	1.0（Very Unconfident）	是	1	1	5.0（Very Confident）	A	✅

Case #100：BPPV（头晕）

GPT 提问（Doctor / GPT）

> Do the dizziness episodes occur when she changes head position, such as turning over in bed or looking up?

中文

头晕是否发生在改变头部位置的时候，例如翻身、躺下、坐起或者抬头？

Patient Ground Truth（Atomic Facts）

隐藏事实共有 11 条，其中与该问题相关的是：

Fact ID	English	中文	是否命中
5	Episodes usually occur immediately after lying down or suddenly sitting up from a reclined position.	发作通常发生在躺下或从半躺姿势突然坐起时。	✅
6	Episodes also sometimes happen when she suddenly turns her head to look at something.	有时突然转头看东西时也会发作。	✅

Claude 返回（Patient Response）

> Episodes usually occur immediately after lying down or suddenly sitting up from a reclined position.

> Episodes also sometimes happen when she suddenly turns her head to look at something.

对比查看以后发现程序是正常运行的，没有并没有出错。也许可以考虑强制达到very confidence然后测试模型医疗水平上限。

July 23th Zoe

Experiment2

1. 读配置：从 `mediq_experiment/config.yaml` 读 doctor_a=gpt-5.4、doctor_b=claude-sonnet-4-6，以及 case 范围 id 100–199（和 Experiment 1 完全同一批100个case）
2. 对每个 case：
    - 取原始 `facts` 字段，去掉"1. "这种编号前缀（`normalize_facts`）

    - 过滤掉"physician orders"类事实(`filter_scorable_atomic_facts`),得到"可用事实池"
3. 对每个 k = 1, 2, 4, 8, all,重复3次:
    - 从事实池里随机抽 k 条(k超过池子大小时自动用全部)
    - 拼成一个静态 prompt(事实列表 + 问题 + 选项 + "只回答字母"指令)
    - 同一次抽样结果,同时喂给 doctor_a 和 doctor_b(配对比较,不是各自独立抽样)
    - 用 `extract_letter` 解析回复里的字母(能处理markdown加粗等格式)
    - 和标准答案比对,记录对错
4. 输出:
    - 逐条记录 → `mediq_experiment/outputs/fact_coverage_experiment/records_nXXX.jsonl`
    - 汇总准确率表 → 同目录下 `..._summary.json`

100个case跑完:

k	Agent A (gpt-5.4)	Agent B (claude-sonnet-4-6)
1	0.5800	0.5767
2	0.6567	0.6300
4	0.7333	0.6667
8	0.8600	0.7700
all	0.9133	0.8300

几个观察:

- 两个agent都随k单调上升
- Agent A 在每一档都领先 Agent B,而且差距随k增大而扩大(k=1时只差0.3个点,几乎打平;到all时差距拉大到8.3个点)
- 和 Experiment 1 对比:Exp1 里 Final Accuracy 是 A=0.79 vs B=0.67(差12个点)。Exp2 在 k=all(完整信息)时是 A=0.91 vs B=0.83(差8.3个点)——说明就算给两个agent完全一样、完整的信息,Agent A 依然更强,这部分差距是真实的推理能力差异,不能全部归因于"谁更会通过对话收集信息"

July 31 by Zoe

AI Hospital(506例里随机抽100):

- Doctor / Patient / Reporter 三方问诊循环——prompt、终止条件、信息不对称设计(病人只知道病史,检查员才知道查体/化验)全部照抄原版代码
- Judge 照抄原版的5维度打分 rubric,二值化成 Final Accuracy(每项 A-D 四档,A 或 B 算 correct=True,C 或 D 算 False,作为 Final Accuracy)
- 交叉诊断:把 A 的问诊记录冻结喂给 B 出诊断,反之亦然

3MDBench(2996例里随机抽100,跳过图片只用文本):

- Doctor / Patient 两方对话(没有检查员),原版 prompt 和 34 类固定诊断列表

- 诊断抽取 + 判分（因为诊断是闭集，判分比 AI Hospital 简单，直接让文本 judge 比较抽取出的诊断 vs. 34 类里的 ground truth 标签，直接语义匹配）(原版需要照片)
- 同样加了交叉诊断

（沿用上周ray的 跑 dialogue → 交叉诊断 → 算 Final Accuracy/Fact Coverage的评测脚本）

3MDBench

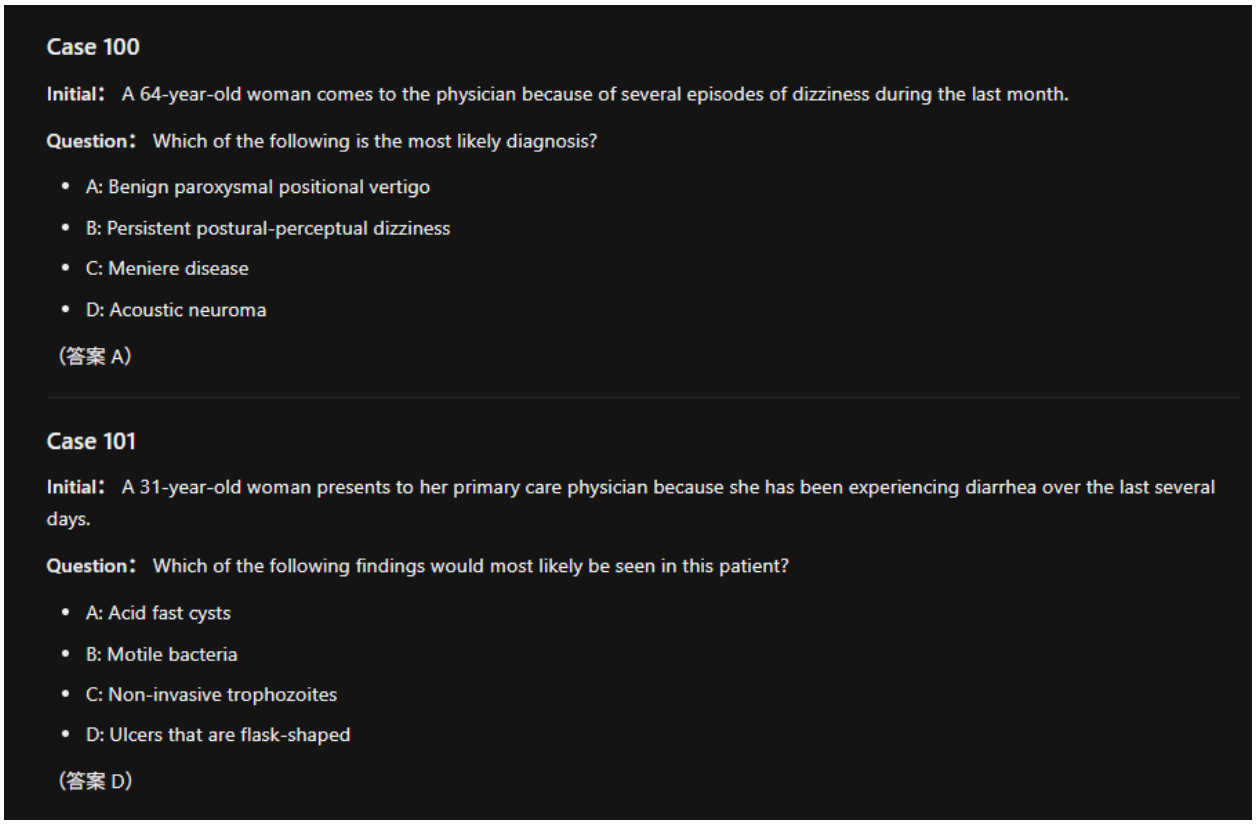
Dialogue | Diagnosis	Agent A（gpt-4o-mini）	Agent B(Claude Haiku 4.5)
Agent A(GPT)	0.57	0.63
Agent B(Claude)	0.67	0.69

AI Hospital

Dialogue | Diagnosis	Agent A（gpt-4o-mini)	Agent B ((Claude Haiku 4.5)
Agent A (GPT)	0.29	0.32
Agent B(Claude)	0.33	0.28

AI Hospital：两模型准确率接近（29% vs 28%），交叉诊断普遍比自己诊断准
3MDBench：Claude Haiku 明显强于 GPT-4o-mini（63-69% vs 57-67%）

Updated: Aug 7, 2026 by Ray
传给模型的内容：



Case 100

Initial： A 64-year-old woman comes to the physician because of several episodes of dizziness during the last month.

Question： Which of the following is the most likely diagnosis?

- A: Benign paroxysmal positional vertigo
- B: Persistent postural-perceptual dizziness
- C: Meniere disease
- D: Acoustic neuroma

(答案 A)

Case 101

Initial： A 31-year-old woman presents to her primary care physician because she has been experiencing diarrhea over the last several days.

Question： Which of the following findings would most likely be seen in this patient?

- A: Acid fast cysts
- B: Motile bacteria
- C: Non-invasive trophozoites
- D: Ulcers that are flask-shaped

(答案 D)

Case 100: 2/11
Case 101: 2/7
fact-coverage针对MediQ有缺陷，initial的问题部分没有纳入计算范围，已经更新，后续需要重跑fact-coverage得到新的数据

Updated: Aug 7, 2026 by Zoe

3MDBench 100-case 最终结果（DeepSeek-V4-Pro vs GPT-5.6-luna）

	Diagnosis A (DeepSeek)	Diagnosis B (luna)
Dialogue A（DeepSeek问诊）	67%（自己诊断）	69%（交叉）
Dialogue B（luna问诊）	70%（交叉）	69%（自己诊断）

Agent A: gpt-5.4
Agent B: sonnet-4.6
Agent C: gpt-5.6-luna
Agent D: Qwen-3.5-4B-OptiQ-4bit
Agent E: Meta-Llama-8B-Instruct-4bit

Dialogue | Diagnosis	Agent A	Agent B	Agent C	Agent D	Agent E
Agent A	0.79	0.6	0.85	0.47	0.44
Agent B	0.69	0.67	0.76	0.42	0.44
Agent C	0.74	0.59	0.81	0.4	0.43
Agent D	0.75	0.58	0.75	0.49	0.35
Agent E	0.77	0.79	0.85	0.41	0.53

AI Hospital 100-case 最终结果（DeepSeek-V4-Pro vs GPT-5.6-luna）

	Diagnosis A (DeepSeek)	Diagnosis B (GPT-5.6-luna)
Dialogue A（DeepSeek问诊）	40%（自己诊断）	48%（交叉）
Dialogue B（luna问诊）	69%（交叉）	51%（自己诊断）

AI Hospital 四档评分表（DeepSeek-V4-Pro vs GPT-5.6-luna，n=100 每格，A=3/B=2/C=1/D=0）

	Diagnosis A (DeepSeek)	Diagnosis B (luna)
Dialogue A（DeepSeek问诊）	A:14 B:26 C:36 D:24 mean=1.30, var=0.97, sd=0.99	A:12 B:36 C:33 D:19 mean=1.41, var=0.86, sd=0.93
Dialogue B（luna问诊）	A:24 B:45 C:19 D:12 mean=1.81, var=0.87, sd=0.94	A:13 B:38 C:31 D:18 mean=1.46, var=0.87, sd=0.93

观察

AI Hospital：失败模式

1. 主诊断答对，但漏掉并发症被扣分（最常见的"部分对"情况）
    - Case 1963：模型说"子宫肌瘤"，真实诊断是"子宫平滑肌瘤+子宫内膜增生+子宫内膜息肉"——核心病灶抓对了，但AI Hospital的病例经常是2-4个共存诊断，只抓住主要那个就会被judge打成B/C档，不能算"完全对"
2. 解剖位置相邻/机制相似的病，容易混
    - Case 1822：真实"跟腱止点炎" vs 模型说"跟腱后滑囊炎"——都在跟腱附近，症状高度重叠，这种解剖学上"邻居"病很容易搞混
3. 一个症状对应多个可能病因，模型选错分支
    - Case 1217："眼黄尿黄"（黄疸）可能是肝炎也可能是胰头癌，DeepSeek 直接判成"胰头癌"，luna 干脆不敢下结论——都没能问出足够信息去正确区分感染性 vs 肿瘤性黄疸
4. 真实病历过于复杂，问诊时间根本不够覆盖
    - Case 1896：真实诊断有 17 条（感染性休克+多器官衰竭+糖尿病+心脏病+肿瘤等），10-20轮问诊物理上不可能问全，模型只能抓大放小
5.（最有意思的一个）"完美主义"式拒诊——明明数据够用，模型非要等"标准答案"式的检查
    - Case 1908（甲状腺结节）：两个模型都判错，都说"无法诊断"。追进对话才发现：医生反复要求做甲状腺超声+TI-RADS分级，但病历原始数据其实是甲状腺增强CT（不是超声），检查员如实回答"没有超声报告"。医生因此卡住，坚持"缺少决定性的超声报告，无法诊断"——但CT报告里其实已经写了"恶性伴...淋巴结转移不除外"，医生压根没有转而利用手头已有的CT结果，白白放弃了本该能推出的正确诊断。这是个协议设计暴露出的真实弱点：模型对检查方式有固定预期，预期落空时不会灵活转向已有的替代数据。

一半的case根本没等到病人主动说"结束"，是被10轮上限硬切断的

实际用的轮数	case数
10（触顶）	49
9	13
8	11
7	10
6	10
5	5
4	2

平均 8.6 轮，**49/100 是被10轮上限截断的**（不是病人主动觉得问够了），另外51个case是病人自己判断"信息够了"提前结束。

让我再看一下，这个"被截断"跟"多诊断共存"（前面说的复杂case）是不是相关——如果复杂case更容易撞上限，就能直接印证"10轮不够问全"这个猜测：

3MDBench：失败模式

1.（最强的一个规律）皮肤病之间的"长得像"混淆，很可能是丢图片导致的
    - Case 2404：真实"疱疹"，两个模型都判成某种"湿疹"（DeepSeek: dyshidrotic eczema，luna: eczema）。追对话发现，病人描述"手指上透明液体的小水疱，又痒又痛"——这个描述本身对疱

疹和湿疹都说得通，只有看图（水疱是否成簇、有无破溃结痂这些视觉特征）才容易鉴别。医生也只问了3个问题就下结论，没有深挖"有没有反复发作""发疱前有没有刺痛感"这类能帮助鉴别疱疹的病史线索
    - Case 803、287：真实"湿疹" vs 模型说"接触性皮炎"/"过敏性接触性皮炎"——同样是外观相似的皮肤病家族内互相误判
    - 这几个案例都直接指向我们最早做的那个决定——跳过图片、只跑文本。3MDBench 原版设计就是要配图片鉴别的，我们砍掉图片，等于把这类"靠肉眼形态鉴别"的皮肤病判断难度人为拉高了
2. 判分口径上的"假阴性"——模型答案不算错，但被判错
    - Case 943：真实"牙结石"，两个模型都答"牙结石+牙龈炎"——临床上牙结石确实常伴发牙龈炎，这个补充诊断不算错，但因为不完全等于单一标签"牙结石"，被判incorrect。这是判分机制偏严格导致的，不完全是模型的问题
3. 相对好答的案例，是"机制单一、体征具体"的病
    - Case 821（甲营养不良）、2130（脓肿）——症状描述直给、病因单一，两个模型都能稳定答对，跟 AI Hospital 里"单一骨折"这类简单case答对的规律一致

两个数据集的共同规律

越是"教科书式单一病因"的case，两个模型都答得对；越是"多系统共病"或"需要视觉/特定检查确认"的case，两个模型经常在同一个case上一起翻车——说明失败很大程度上是任务本身的固有难度（复杂共病、缺失关键模态），不完全是模型能力差距，这跟你之前问的"为什么交叉诊断有时候更准"也能对上：当任务难度是瓶颈时，换哪个模型都好不了多少，这时候交叉诊断的"换个角度看"反而比模型本身的能力差异更能决定结果。

100个case里，47个是"多个共存诊断"（真实诊断包含2个及以上病症），53个是单一诊断

关键发现：拿"满分A"的概率，在多诊断case上明显更低——两个模型都是

	单一诊断case (n=53)	多诊断共存case (n=47)
DeepSeek 拿A档比例	11/53 = 20.8%	3/47 = 6.4%
luna 拿A档比例	10/53 = 18.9%	3/47 = 6.4%

两个模型在多诊断case上，拿满分A的概率都掉到了单一诊断case的1/3左右——这个是稳定、一致的规律，你举的那个case（1963，抓住主诊断但漏了共病）是这个规律的典型代表。

但"二元正确率"（A+B算对）没有这么一致，甚至有一个模型反过来了

	单一诊断 correct_rate	多诊断 correct_rate
DeepSeek	34%	**47%**（反而更高）
luna	57%	45%（符合预期，更低）

原因：多诊断case虽然很难拿A，但也不容易跌到D——因为judge只要模型抓住了**主要**诊断，哪怕漏了次要共病，通常也会给B档（"相当部分一致"），而B在我们的二值化规则里算"对"。所以"主诊断对但漏共病"这个现象，**在A档层面非常明显、非常常见（漏共病=几乎拿不到A）**，但因为B档兜底，不会直接拖累到二元正确率——DeepSeek 甚至因为单一诊断case本身答得不够稳（拿了不少D），多诊断case反而靠B档兜底显得"正确率更高"。

Updated: Aug 5, 2026 by Ray

```
"models": {
  "doctor_a": "openai/gpt-5.4",
  "doctor_b": "anthropic/claude-sonnet-4-6",
  "doctor_c": "openai/gpt-5.6-luna",
  "doctor_d": "huggingface/mlx-community/Qwen3.5-4B-OptiQ-4bit",
  "doctor_e": "huggingface/mlx-community/Meta-Llama-3.1-8B-Instruct-4bit",
  "patient": "anthropic/claude-haiku-4-5",
  "judge": "anthropic/claude-haiku-4-5"
},
```

Mean_fact_coverage:

A：0.074187
B：0.054847
C：0.0815569
D：0.123398
E：0.326393

对话轮数：

```
doctor_a: n=100  avg_turns=3.37  min=0 max=30  approx_timeout_rate=0.07
doctor_b: n=100  avg_turns=1.73  min=0 max=30  approx_timeout_rate=0.02
doctor_c: n=100  avg_turns=3.51  min=0 max=30  approx_timeout_rate=0.07
doctor_d: n=100  avg_turns=5.67  min=0 max=30  approx_timeout_rate=0.10
doctor_e: n=100  avg_turns=28.57  min=2 max=30  approx_timeout_rate=0.93
```

Agent A在Interactive模式中准确率最高，且准确率随fact命中率上升
Agent B使用E的对话历史记录上准确率显著升高，但其他的fact命中率并没有提升准确率
Agent C使用A和E的对话历史记录上准确率都在0.85，但是两者的fact命中率差值＞2，Interactive模式略差于一次性提供对话
Agent D自身Interactive准确率最高，并未受fact命中率影响
Agent E自身Interactive准确率最高，并未受fact命中率影响

Updated: Aug 1, 2026 by Ray
后续三个Agent进行更新：
Agent C: gpt-5.6-luna
Agent D: Qwen3.5-4B
Agent E: Llama-3.1-8B-Instruct
在等待Llama的审批，Zoe同步申请，备用方案DeepSeek-R1-0528-Qwen3-8B

Updated: Jul 30, 2026 by Ray

Role	Hugging Face ID	简称	规模	定位 / 相对优点
Doctor C	`google/gemma-2-9b-it`	Gemma 2 9B Instruct	~9B	通用指令模型；注释写明 **暂代 Llama-3.1-8B（等 HF 权限）**；同档里参数略大，指令跟随与对话质量通常较好
Doctor D	`Qwen/Qwen2.5-7B-Instruct`	Qwen2.5-7B Instruct	~7B	开源通用强模型；多语/指令表现稳；**无 Llama 门控**，下载更省事；单卡跑得动
Doctor E	`aaditya/Llama3-OpenBioLLM-8B`	OpenBioLLM-8B	~8B	**医学域微调**（Llama3 底座）；相对前两个更贴 clinical / biomedical 问答，适合和通用开源模型做对照



src仅保留MediQ的源码，smart_trial能完全脱离独立运行

2. 核心试验流程（两阶段 SMART）

当前实现是 **两阶段**（Stage 1 → R1 → Stage 2 → R2 → 结局），在 `TrialOrchestrator` 中编排：

阶段	轮次	内容
Stage 1	默认 4 轮	信息收集；随机分配 A1a/A1b/A1c 三种问诊策略
R1	—	LLM Judge 对病史充分性打分（0–10），≥6 为 responder
Stage 2	默认 6 轮	根据 R1 **条件重随机化** A2 臂；在此阶段给出最终诊断
R2	—	从医生输出的 `[CONFIDENCE: 0.XX]` 或 Judge 回退估计置信度
Outcome	—	诊断正确性、危险建议、管理合理性等

Stage 2 随机化池（`TrialRandomizer`）：

- **Responder**（R1≥6）：`{A2a, A2b, A2c}` 三等分
- **Non-responder**：`{A2a, A2b}` 二等分（A2c 仅对 responder 开放）

三种运行模式（`run.mode`）：

- `smart_random`：完整 SMART 随机试验
- `smart_grid`：强制指定 (A1, A2) 组合，用于 3×3=9 条静态路径网格评测
- `baseline`：无策略臂、单阶段连续对话（对照组）

Stage 1由Opening+4轮

目前Stage1&2改为固定4&6轮对话，减少变量影响以便于和baseline进行对比，后期可改为自适应：R1分数达标后自动early stop提前进入R2

6 个 Strategy 的逻辑

分两层理解：

Stage 1 的三个 arm（信息收集风格）

Arm	风格	逻辑
A1a	全面系统问诊	把 OPQRST、既往史、用药、社会史都问一遍
A1b	聚焦主诉	只围绕病人说的主要症状深挖
A1c	红旗优先	先问有没有危险信号，再展开

三个 arm 代表三种不同的**问诊哲学**，互斥，Stage 1 随机分一个。

Stage 2 的三个 arm（检索策略）

Arm	风格	逻辑	谁能用
A2a	每轮都检索	不管有没有把握都查 BM25	所有人
A2b	置信度条件检索	只有 confidence < 0.7 才查	所有人
A2c	纯参数推理	完全不检索，靠模型自身知识	**仅 responder**

StageJudge（`core/judge.py`）

- **R1**：五维 rubric（OPQRST、red flags、PMH、用药过敏、社会家族史）→ JSON 打分
- **R2**：优先用医生自报 confidence；不足时用 Judge LLM 回退
- **Outcome**：对比 ground truth，判断 `diag_correct`、`dangerous_advice`、`appropriate_management`

离线Q-learning：

三个关键量：

符号	含义	怎么算
`pseudo_y1[i]`	Stage 2 最优值	在 **观测到的** H_2 上，对可行臂池 `pool(R1)` 取 `argmax_a2` $\hat{Q}_2$
`pseudo_y0[i]`	全策略最优值	对每个候选 a_1，构造 **反事实** $H_2(a_1)$，再 `max_a2` $\hat{Q}_2$；最后在 3 个 a_1 里取最大
`value` (= $\hat{V}(\pi)$)	估计最优策略价值	`mean(pseudo_y0)`

设计是标准的 backward induction（向后归纳）：



MediQ Interactive 策略改进：

BASIC

就是最简单的设定：告诉模型"你可以选择问一个问题，或者直接给答案"，完全靠模型自己判断什么时候问、问什么。没有任何额外机制辅助。

BEST

是他们实验了很多种"让模型更好地判断自己有没有把握"的策略之后，表现最好的那个组合。具体是三个东西叠加：

- **Scale Abstention**：不是让模型说"我有没有把握"，而是让它在一个1-5的量表上选，"非常有把握/比较有把握/一般/不太有把握/完全没把握"，更细粒度
- **Rationale Generation**：在做判断之前先让模型写一段推理，解释自己为什么有或没有把握，这样模型的置信度估计更准
- **Self-Consistency**：同一个问题问模型3次，取平均或多数投票，减少随机性

这三个叠加之后，模型能更准确地知道"我现在信息够了"还是"我还需要再问"，所以问的问题更有针对性，最终准确率更高。

Experimental Framework: Fact-Tier Sufficiency

Objective: To empirically test the hypothesis that "Most Important" clinical facts are sufficient for diagnostic accuracy, and to identify if current agents suffer from a retrieval bottleneck rather than a reasoning deficit.

Phase 1: The Gold Standard Set (Control)

- **Definition:** Establish a "Tier-1" dataset for your test cases where an expert human or a high-parameter model (e.g., GPT-5.6-Luna) has manually curated the "Most Important" clinical facts.
- **Validation:** Use this set as the benchmark "All Facts" baseline for subsequent comparisons.

Phase 2: The Agentic Retrieval Test (Treatment)

- **Protocol:** Configure the Dialogue Agent to perform the standard clinical interview.
- **Variable Variations:**
    - **Variation A (Control):** The agent performs standard, unconstrained clinical dialogue.
    - **Variation B (Experimental):** The agent is explicitly prompted to pause and extract/summarize "Most Important" facts at specific turns (e.g., turns 3, 6, and 10).
- **Comparison:** Compare the accuracy of the Diagnosis Agent when fed the Gold Standard Set vs. the Agent-Extracted Facts from Variation B.

Phase 3: Metrics for Validation

- **Primary Metric:** Diagnosis Accuracy (Goal: Achieve ≥85% of "All facts" accuracy using only Tier-1 "Most important" facts).
- **Secondary Metric:** Retrieval Recall (How many of the "Most Important" facts from the Gold Standard were present in the Agent-Extracted set from Variation B?).
- **Efficiency Metric:** Turn-count required to reach the extraction threshold.

Proposed Experimental Framework

This framework is designed to systematically evaluate the necessity and retrieval accuracy of critical diagnostic information within agent-patient interactions.

1. **Phase 1: Fact Tier Definition (Ground Truth)**
    - Objective: Establish a gold-standard dataset of "Most Important" facts for target cases.
    - Process: Use expert annotation or high-performing models to tier clinical facts based on their diagnostic weight for specific medical conditions.
2. **Phase 2: Hypothesis 1 Validation (Sufficiency Test)**
    - Objective: Test if "Most important" facts are sufficient for accurate diagnosis.
    - Methodology: Compare Diagnosis Agent accuracy using "All Facts" (Control) vs. "Most Important" facts (Treatment) to verify that the retrieval of non-critical facts does not significantly improve diagnostic outcomes.
3. **Phase 3: Hypothesis 2 Validation (Retrieval Gap)**
    - Objective: Verify if current agents fail to retrieve the most important facts from dialogue.

- Methodology: Compare Gold Standard "Most Important" facts against facts extracted by the current Dialogue Agent. Additionally, implement and test a "Retrieval-Augmented Dialogue" (RAD) prompt constraint to measure potential improvements in targeted fact retrieval.